

Сравнение четкой и нечеткой логики

Базовая задача расчета чаевых

Чтобы проиллюстрировать возможности нечеткой логики, рассмотрим как линейный, так и нечеткий подходы к следующей задаче:

Какова справедливая сумма чаевых для вашего официанта?

Во-первых, решим эту задачу обычным способом, записывая команды MATLAB®, которые описывают линейные и кусочно-линейные зависимости. Затем рассмотрим ту же систему, использующую нечеткую логику.

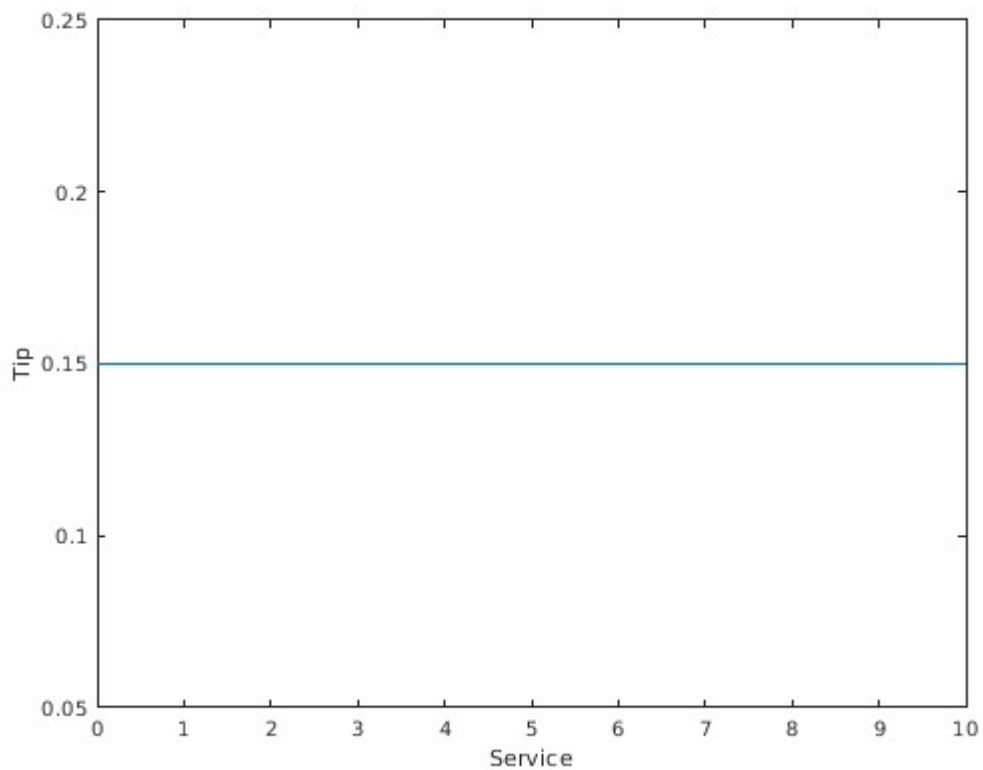
Базовая задача расчета чаевых. Учитывая значение от 0 до 10, которое представляет качество сервиса в ресторане (где 10 превосходно), каковы должны быть чаевые?

Эта задача основана на чаевых, которые обычно практикуются в Соединенных Штатах. В среднем чаевые за еду в США составляют 15%, хотя фактическая сумма может варьироваться в зависимости от качества предоставленной услуги.

Обычный подход

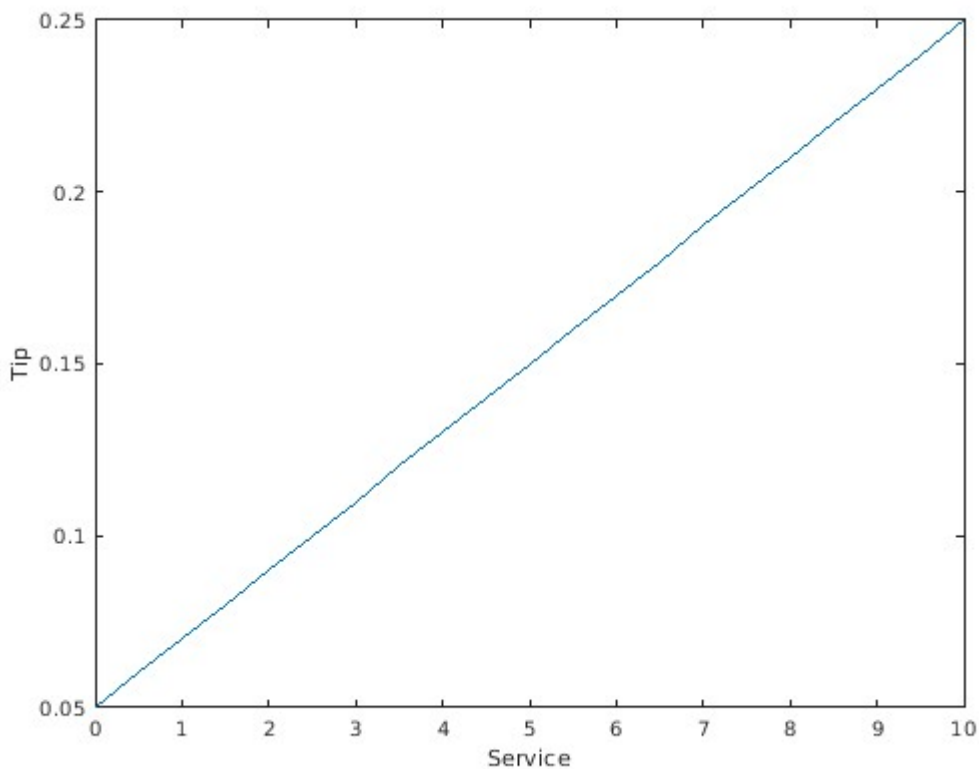
Начнем с самой простой зависимости. Предположим, что чаевые всегда равняются 15% общего счета.

```
service = 0:.5:10;  
tip = 0.15*ones(size(service));  
plot(service,tip)  
xlabel('Service')  
ylabel('Tip')  
ylim([0.05 0.25])
```



Это отношение не включает качество сервиса, так что, необходимо добавить термин в уравнение. Поскольку сервис оценивается по шкале от 0 до 10, вы увеличение совета линейно с 5%, если сервис плох к 25%, если сервис превосходен. Теперь отношение похоже на следующий график:

```
tip = (.20/10)*service+0.05;  
plot(service,tip)  
xlabel('Service')  
ylabel('Tip')  
ylim([0.05 0.25])
```

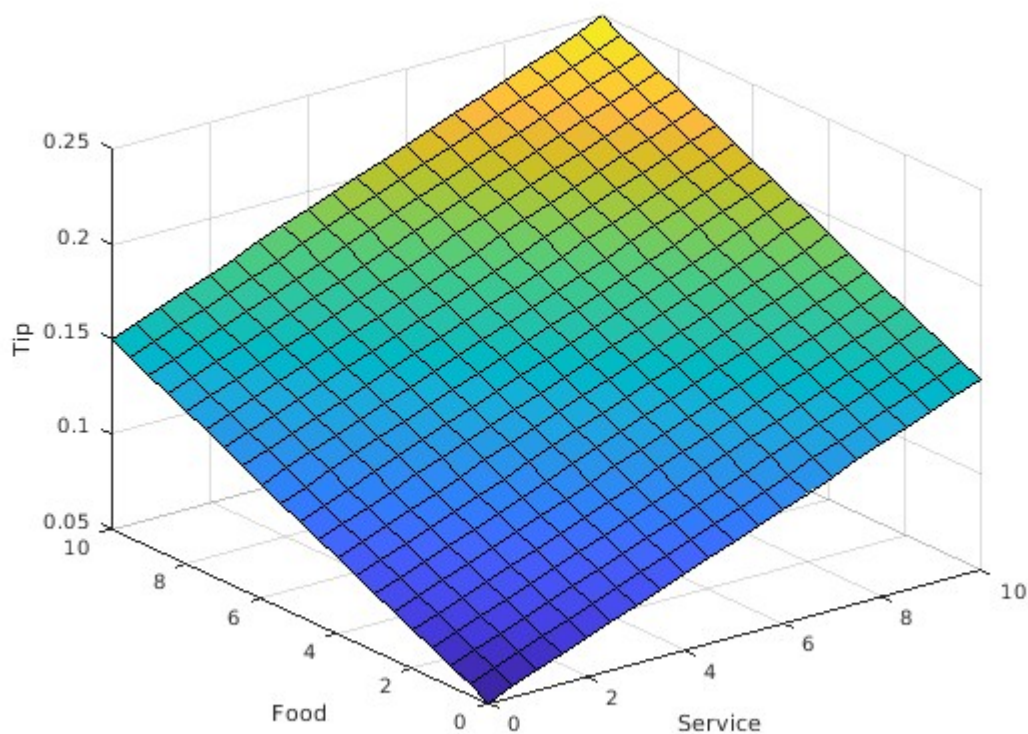


Формула делает то, что вы хотите, чтобы она сделала, напрямую. Тем не менее, вы можете захотеть, чтобы чаевые также отражали качество пищи. Это расширение задачи определяется следующим образом.

Расширенная задача расчета чаевых. Учитывая два значения чисел от 0 до 10 (где 10 превосходно), которые соответственно представляют качество сервиса и качество еды в ресторане, каковы должны быть ваши чаевые?

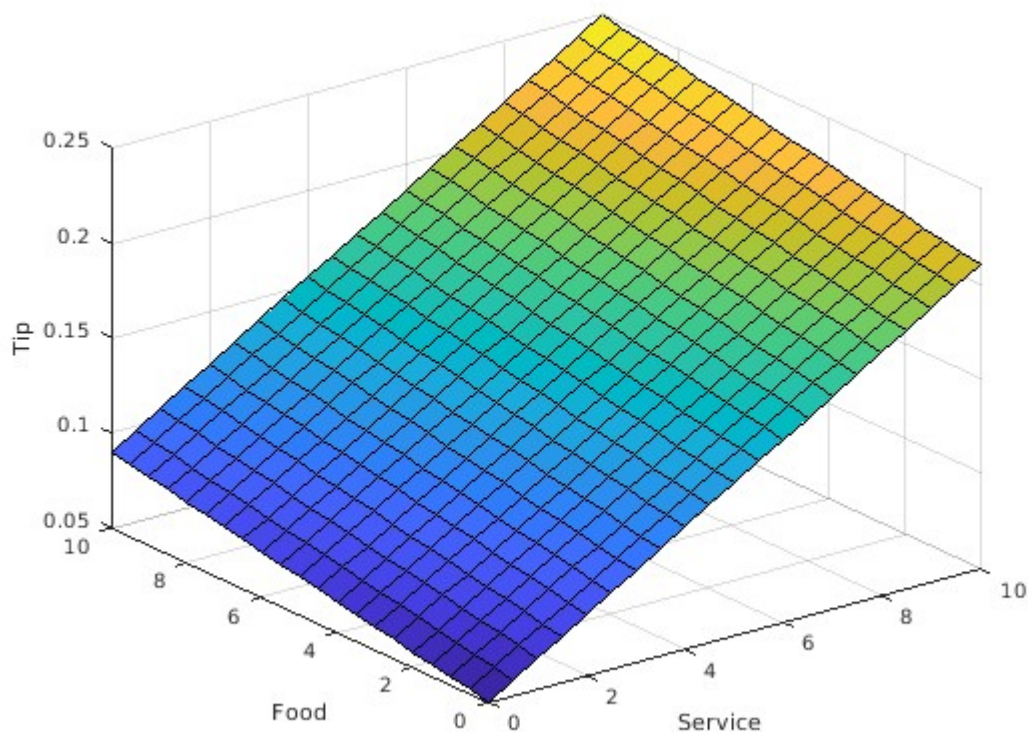
Смотрите, как формула затронута теперь, когда вы добавили другую переменную.

```
food = 0:.5:10;  
[F,S] = meshgrid(food,service);  
tip = (0.20/20).*(S+F)+0.05;  
surf(S,F,tip)  
xlabel('Service')  
ylabel('Food')  
zlabel('Tip')
```



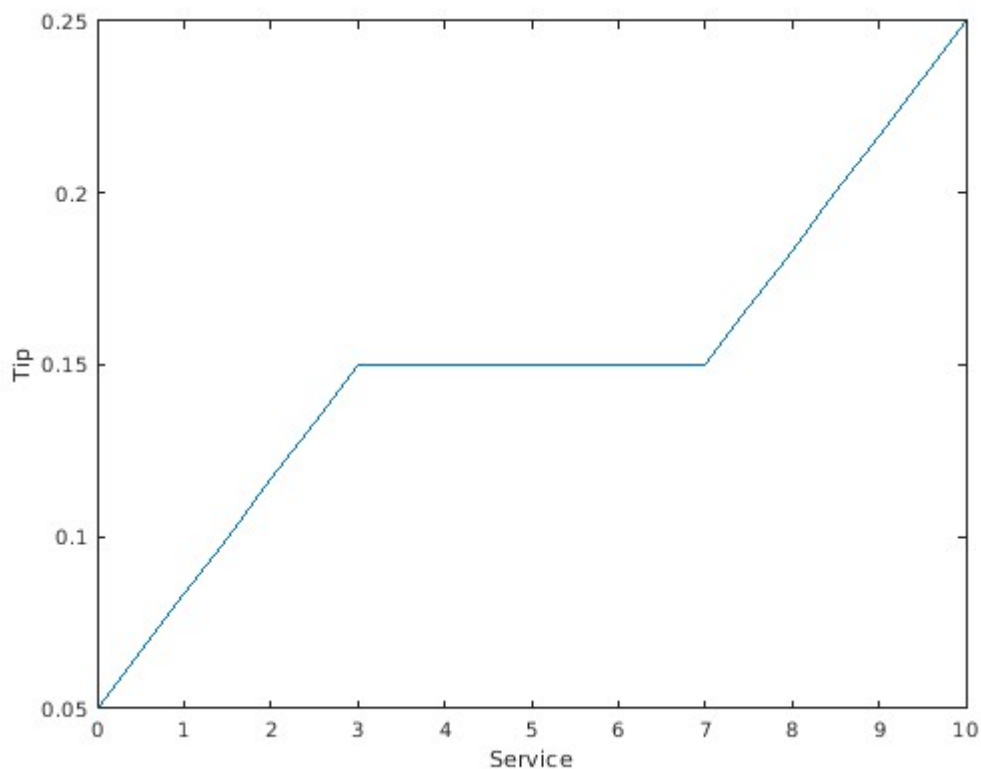
В этом случае результаты выглядят удовлетворительно, но при внимательном рассмотрении, они не кажутся верными. Предположим, что вы хотите, чтобы обслуживание было более важным фактором, чем качество еды. Пусть обслуживание составляет 80% от общего количества чаевых, а еда составляет остальные 20%.

```
servRatio = 0.8;  
tip = servRatio*(0.20/10*S+0.05) + ...  
      (1-servRatio)*(0.20/10*F+0.05);  
surf(S,F,tip)  
xlabel('Service')  
ylabel('Food')  
zlabel('Tip')
```



Реакция все еще слишком равномерно линейна. Предположим, что вы хотите получить более плавный ответ в середине, то есть вы хотите дать 15% чаевых в общем случае, но хотите также увеличить вариацию, если услуга исключительно хороша или плоха. Этот фактор, в свою очередь, означает, что предыдущие линейные отображения больше не применяются. Вы все еще можете использовать линейный расчет с кусочно-линейной конструкцией. Теперь вернемся к одномерной проблеме, рассматривая только сервис. Вы можете создать простое условное назначение чаевых с помощью логического индексирования.

```
tip = zeros(size(service));
tip(service<3) = (0.10/3)*service(service<3)+0.05;
tip(service>=3 & service<7) = 0.15;
tip(service>=7 & service<=10) = ...
    (0.10/3)*(service(service>=7 & service<=10)-7)+0.15;
plot(service,tip)
xlabel('Service')
ylabel('Tip')
ylim([0.05 0.25])
```



Предположим, что вы расширяете этот подход к двум размерностям, где вы вводите качество продуктов снова.

```
servRatio = 0.8;  
tip = zeros(size(S));  
tip(S<3) = ((0.10/3)*S(S<3)+0.05)*servRatio + ...  
    (1-servRatio)*(0.20/10*F(S<3)+0.05);  
tip(S>=3 & S<7) = (0.15)*servRatio + ...  
    (1-servRatio)*(0.20/10*F(S>=3 & S<7)+0.05);  
tip(S>=7 & S<=10) = ((0.10/3)*(S(S>=7 & S<=10)-  
7)+0.15)*servRatio + ...  
    (1-servRatio)*(0.20/10*F(S>=7 & S<=10)+0.05);  
surf(S,F,tip)  
xlabel('Service')  
ylabel('Food')  
zlabel('Tip')
```

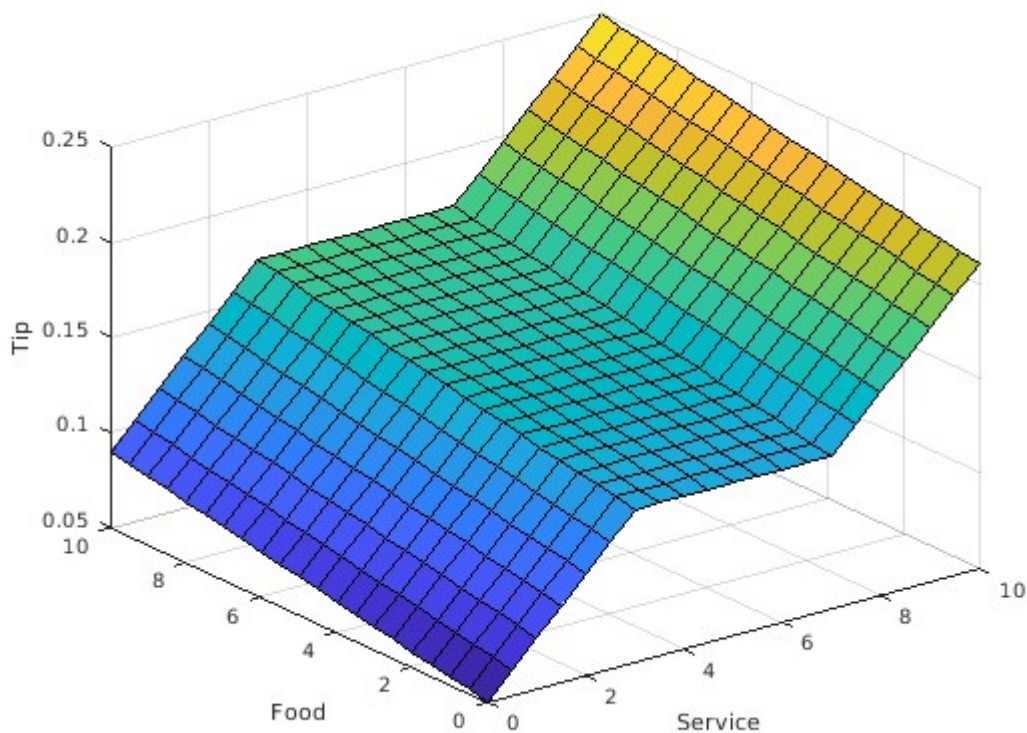


График выглядит хорошим, но функция является достаточно сложной. Некоторым, кто не видел процесс первоначального проекта, даже непонятно как это работает.

Подход нечеткой логики

В общем случае вы хотите получить основы этой проблемы, не принимая во внимание все факторы, которые могли быть произвольными. Если вы составляете список того, что действительно имеет значение в этой проблеме, вы могли закончить со следующими описаниями правила.

Правила задачи о чаевых - фактор обслуживания

- Если сервис плох, то чаевые малы
- Если сервис хорош, то чаевые средние
- Если сервис превосходен, то чаевые щедрые

Порядок, в котором правила представлены здесь, произволен. Это не имеет значения, какие правила на первом месте. Чтобы включить эффект качества продуктов на чаевые, добавьте следующие два правила.

Правила задачи о чаевых - фактор еды

- Если еда является плохой, то чаевые малы
- Если еда восхитительна, то чаевые щедрые

Можно объединить два различных списка правил в один список трех правил как так.

Задача о чаевых - совместные факторы обслуживания и еды

- Если сервис плох, или еда является прогорклой, то чаевые малы
- Если сервис хорош, то чаевые средние
- Если сервис превосходен, или еда восхитительна, то чаевые щедрые

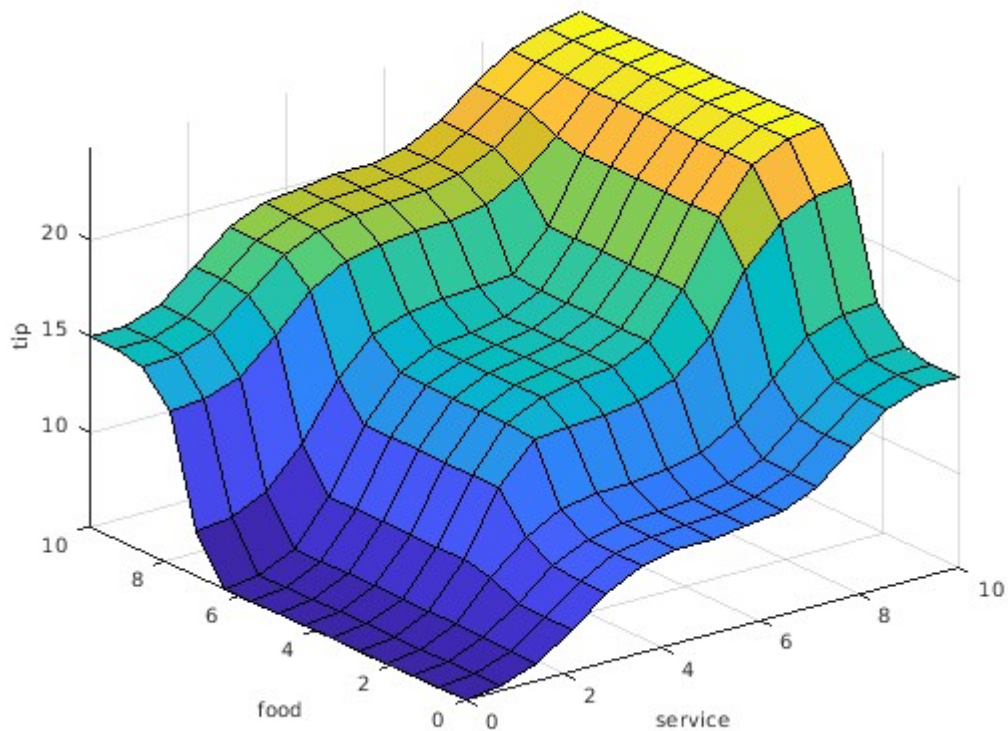
Эти три правила являются ядром вашего решения, и они соответствуют правилам для системы нечеткой логики. Когда вы даете математическое значение лингвистическим переменным (что такое средние чаевые, например), у вас есть полная нечеткая система вывода. Методология нечеткой логики должна также рассмотреть:

- Как все правила объединяются?
- Как я задаю математически, каковы средние чаевые?

Решение задачи

Следующий график представляет систему нечеткой логики, которая решает задачу расчета чаевых.

```
gensurf(readfis('tipper'))
```



Этот график был сгенерирован тремя правилами, которые составляли и сервис и продовольственные факторы.

Наблюдения

Рассматривают некоторые наблюдения о примере до сих пор. Вы нашли кусочно-линейное отношение, которое решило задачу. Это работало, но это было проблематично, чтобы вывести, и когда вы записали его как код, не было легко интерпретировать. С другой стороны, система нечеткой логики основана на некоторых разумных операторах. Кроме того, вы смогли добавить еще два правила в список, который влиял на форму полного выхода, не будучи должен отменить то, что было уже сделано.

Кроме того, при помощи правил нечеткой логики, обслуживание структуры алгоритма разъединяется вдоль довольно чистых линий. Понятие средних чаевых может измениться со дня на день, город в город, страна в страну. Однако базовая логика является тем же самым: если сервис хорош, совет должен быть средним.

Перекалибровка Метода, можно перекалибровать метод быстро путем простого сдвига нечеткого множества, которое задает среднее значение, не переписывая правила нечеткой логики.

Можно переключить списки кусочно-линейных функций, но существует большая вероятность для трудной перекалибровки.

В следующем примере кусочно-линейная задача расчета чаевых переписана, чтобы сделать его более типовым. Это выполняет ту же функцию как прежде, только теперь константы могут быть легко изменены.

```
lowTip = 0.05;
averTip = 0.15;
highTip = 0.25;
tipRange = highTip-lowTip;
badService = 0;
okayService = 3;
goodService = 7;
greatService = 10;
serviceRange = greatService-badService;
badFood = 0;
greatFood = 10;
foodRange = greatFood-badFood;

% If service is poor or food is rancid, tip is cheap
if service<okayService
    tip = (((averTip-lowTip)/(okayService-badService)) ...
        *service+lowTip)*servRatio + ...
        (1-servRatio)*(tipRange/foodRange*food+lowTip);

% If service is good, tip is average
elseif service<goodService
    tip = averTip*servRatio + (1-servRatio)* ...
        (tipRange/foodRange*food+lowTip);

% If service is excellent or food is delicious, tip is
generous
else
    tip = (((highTip-averTip)/ ...
        (greatService-goodService))* ...
        (service-goodService)+averTip)*servRatio + ...
        (1-servRatio)*(tipRange/foodRange*food+lowTip);
end
```

Как со всем кодом, чем больше общности, которая введена, тем менее точным алгоритм становится. Можно улучшить ясность путем добавления большего количества комментариев, или возможно перезаписи алгоритма немного большим количеством самоочевидных способов. Но, кусочно-линейная методология не является оптимальным способом решить этот вопрос.

Если вы удаляете все из алгоритма за исключением трех комментариев, что остается, точно правила нечеткой логики, которые что вы ранее записали.

- Если сервис плох, или еда является прогорклой, совет является дешевым
- Если сервис хорош, совет является средним
- Если сервис превосходен, или еда восхитительна, совет щедр

Нечеткая логика использует язык, который ясен вам, и это также имеет значение к компьютеру, который является, почему это - успешный метод для того, чтобы устранить разрыв между людьми и машинами.

Путем создания уравнений максимально простыми (линейный) вы делаете вещи более простыми для машины, но более сложными для вас. Однако ограничение больше не является компьютером - это - ваша умственная модель того, что делает компьютер. Нечеткая логика позволяет машине работать с вашими настройками, а не наоборот.

Нечеткие системы Используя Fuzzy Logic Designer

Инструменты графического интерфейса пользователя Fuzzy Logic Toolbox

В этом примере показано, как создать нечеткую систему вывода (FIS) для примера добавления подсказки, описанного в Базовой задаче расчета чаевых, с помощью инструментов Fuzzy Logic Toolbox™ UI. Эти инструменты поддерживают только тип 1 нечеткие системы.

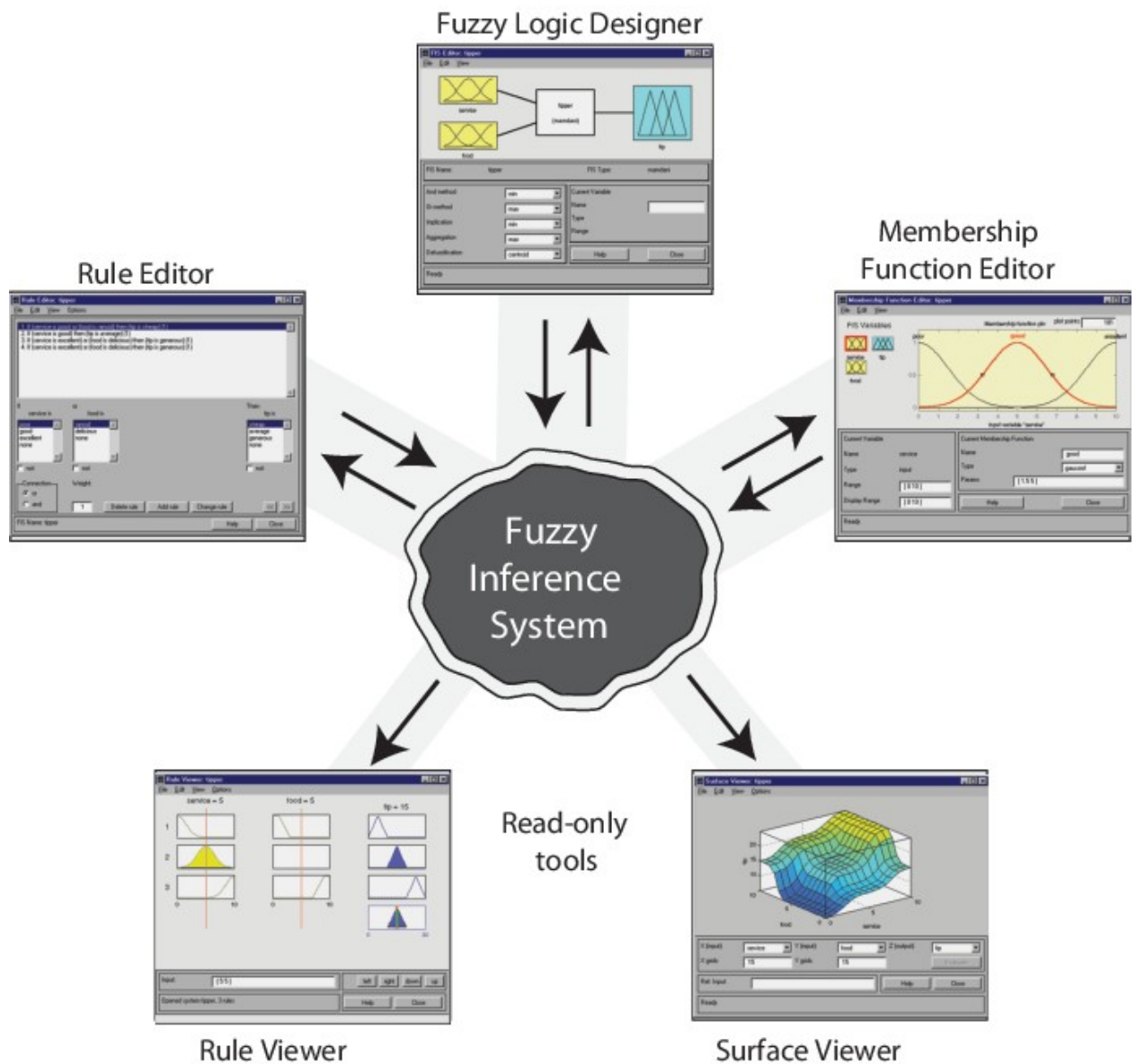
Вы используете следующие инструменты, чтобы создать, отредактировать, и просмотреть нечеткие системы вывода:

- [Fuzzy Logic Designer](#), чтобы обработать высокоуровневые проблемы для системы — Сколько переменных ввода и вывода? Каковы их имена?

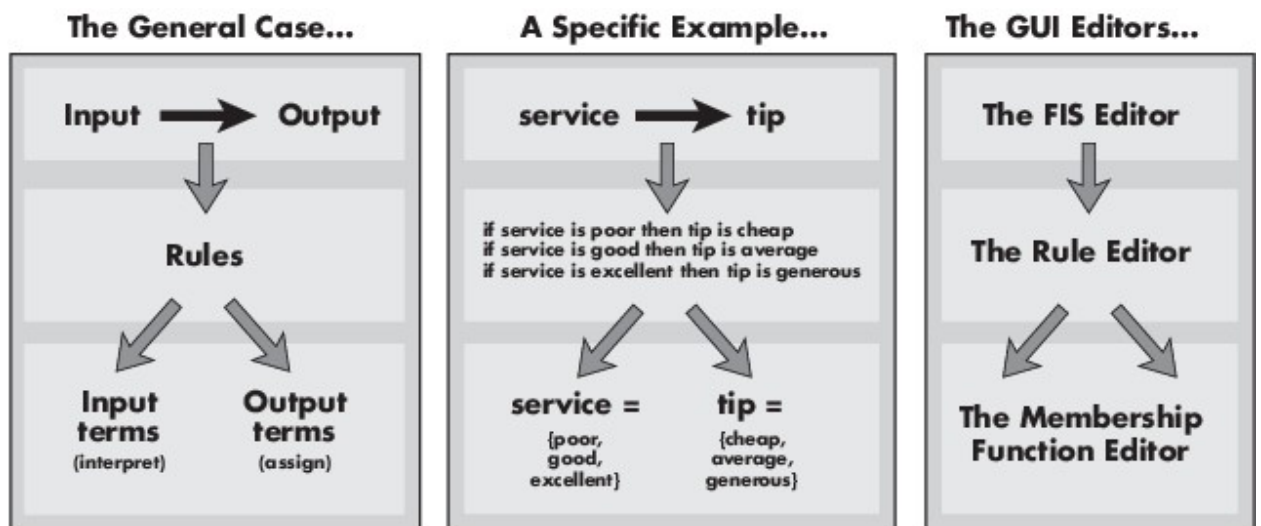
Программное обеспечение Fuzzy Logic Toolbox не ограничивает количество входных параметров. Однако количество входных параметров может быть ограничено доступной памятью о вашей машине. Если количество входных параметров является слишком большим, или количество функций принадлежности является слишком большим, то это может также затруднить, чтобы анализировать FIS использование других инструментов.

- **Membership Function Editor**, чтобы задать формы всех функций принадлежности, сопоставленных с каждой переменной
- **Rule Editor**, чтобы отредактировать список правил, который задает поведение системы.
- **Rule Viewer**, чтобы просмотреть нечеткую схему вывода.
Используйте это средство просмотра в качестве диагностики, чтобы видеть, например, какие правила активны, или как отдельные формы функции принадлежности влияют на результаты
- **Surface Viewer**, чтобы просмотреть зависимость одних из выходных параметров на любом или двух из входных параметров; то есть, это генерирует и строит выходную карту поверхности для системы.

Эти UIs динамически соединяются, в этом изменения вы делаете к FIS использование одного из них, влияете на то, что вы видите на любом из других открытых UIs. Например, если вы меняете имена функций принадлежности в Редакторе Функции принадлежности, изменения отражаются в правилах, показанных в Редакторе Правила. Можно использовать UIs, чтобы считать и записать переменным обоим в MATLAB® рабочая область и к файлу (средства просмотра только для чтения могут все еще обмениваться графиками с рабочей областью и сохранить их в файл). У вас могут быть любые из них, открываются для любой данной системы или имеют несколько редакторов, открытых для любого количества нечетких систем.



Следующий рисунок показывает, как основные компоненты FIS и этих трех редакторов совмещаются. Эти два средства просмотра исследуют поведение целой системы.



В дополнение к этим пяти первичным UIs тулбокс включает графический [Neuro-Fuzzy Designer](#), который вы используете, чтобы создать и анализировать Sugeno-тип адаптивные нейронечеткие системы вывода.

Fuzzy Logic Toolbox UIs не поддерживает создание FIS использование данных. Если вы хотите использовать данные, чтобы создать FIS, используйте один из следующих методов:

- genfis сгенерировать Sugeno-тип FIS. Затем выберите **File> Import in Fuzzy Logic Designer**, чтобы импортировать FIS и выполнить нечеткий вывод, как описано в [Fuzzy Logic Designer](#).
- Нейроадаптивные методы изучения, чтобы смоделировать FIS, как описано в [Нейроадаптивном Изучении и ANFIS](#).

Если вы хотите использовать переменные рабочего пространства MATLAB, используйте интерфейс командной строки вместо **Fuzzy Logic Designer**. Для примера смотрите Сборку Нечеткие Системы в Командной строке. См ниже.

Базовая задача расчета чаевых

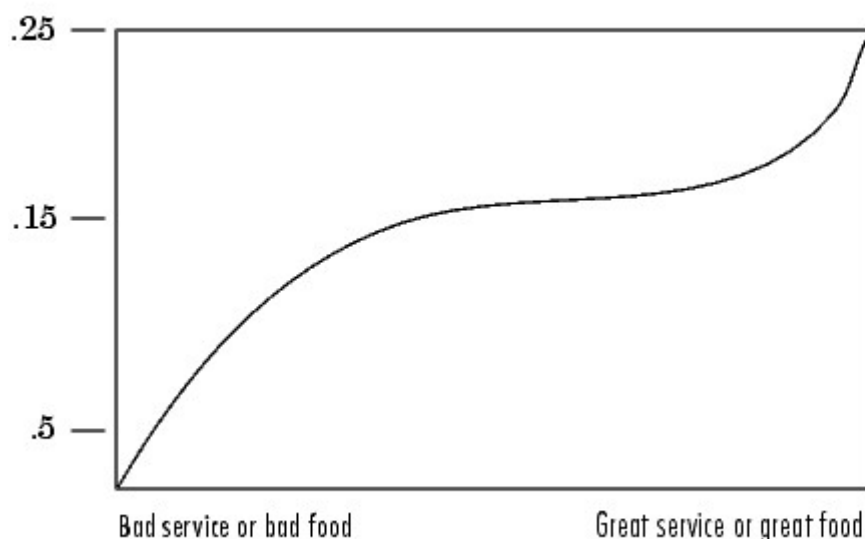
Этот пример создает Mamdani нечеткое системное использование вывода на 2D входе, задаче расчета чаевых с одним выходом на основе добавления подсказки методов в США. В то время как пример создает Mamdani FIS, используемые методы применяются к созданию систем Sugeno также.

Учитывая номер между 0 и 10, который представляет качество сервиса в ресторане (где 10 превосходно), и другой номер между 0 и 10, который представляет качество еды в том ресторане (снова, 10 превосходно), каковы должны быть ваши чаевые?

Начальная точка должна записать три золотых правила добавления подсказки:

1. Если сервис плохой, или еда - тухлая, то чаевые будут маленькими.
2. Если сервис хороший, то чаевые будут средними.
3. Если сервис превосходен, или еда восхитительна, тогда чаевые будут большими.

Примите, что средние чаевые составляют 15%, щедрый совет составляет 25%, и дешевый совет составляет 5%.



Числа и форма кривой подвергаются локальным традициям, культурному смещению, и так далее, но три правила обычно универсальны.

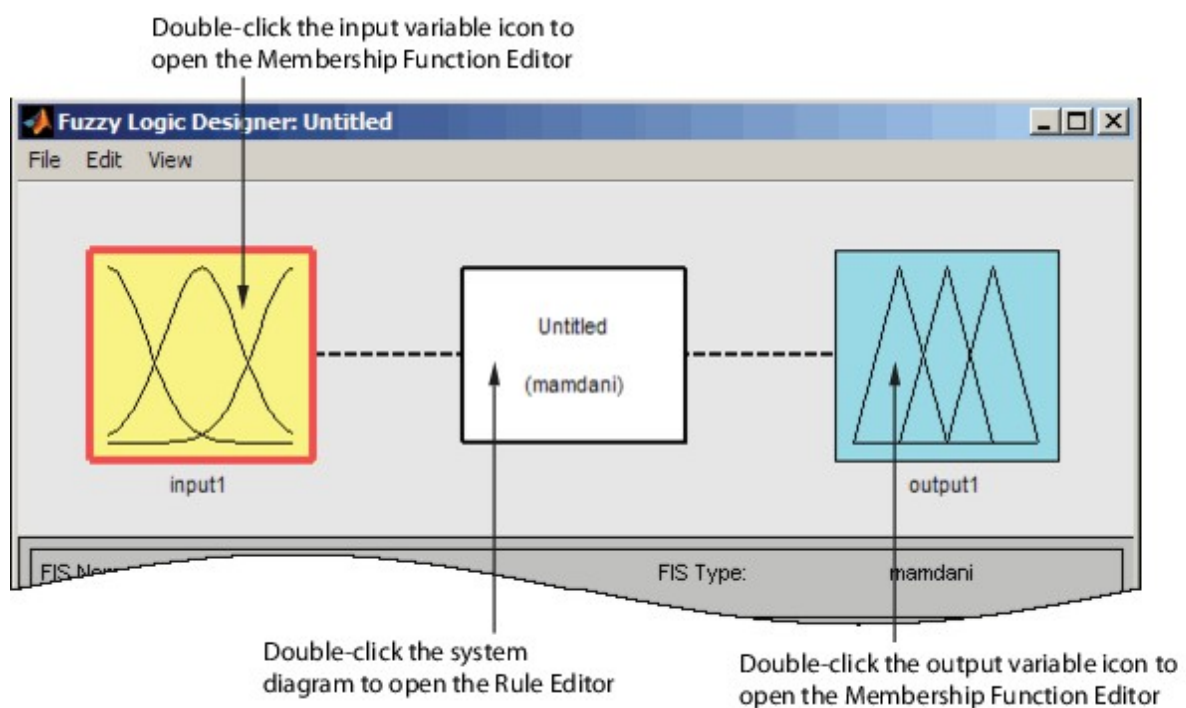
Теперь, когда вы знаете правила и имеете идею того, на что должен быть похожим выход, используйте инструменты UI, чтобы создать нечеткую систему вывода для этого процесса принятия решений.

Fuzzy Logic Designer

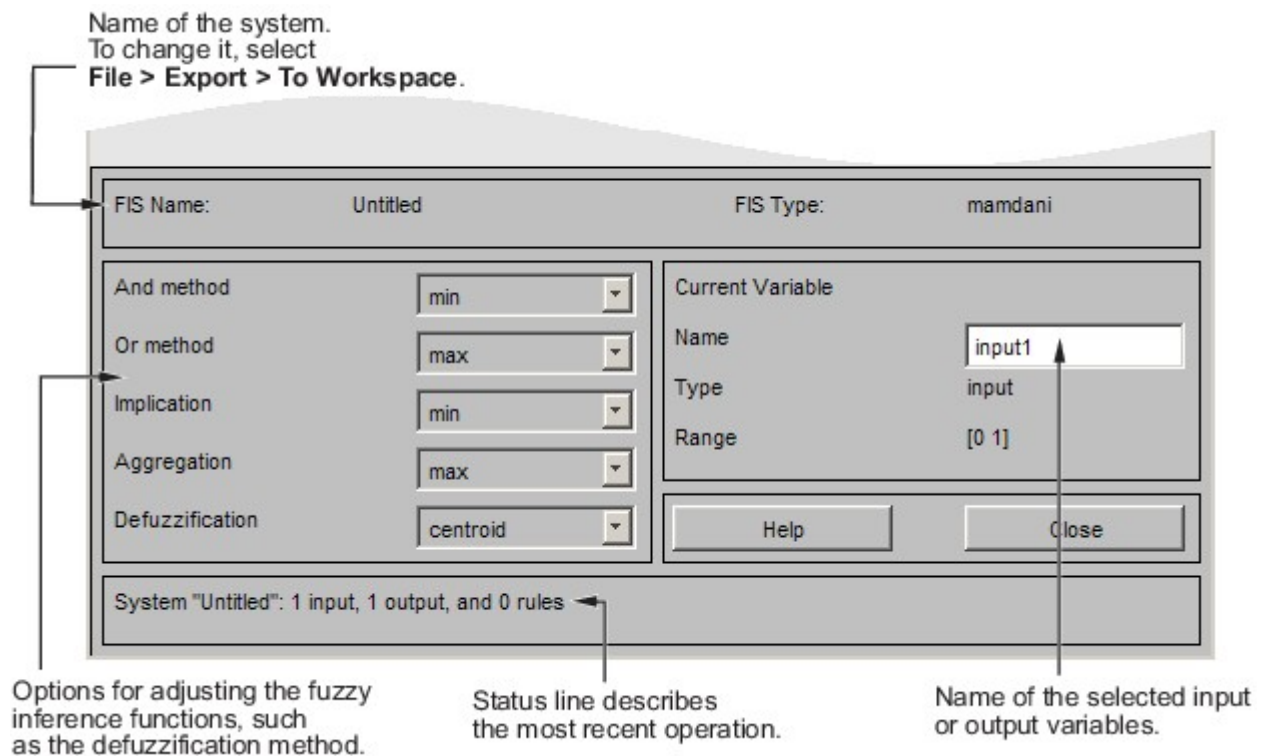
Информация об отображениях приложения **Fuzzy Logic Designer** о нечеткой системе вывода. Чтобы открыть **Fuzzy Logic Designer**, введите следующую команду в подсказке MATLAB:

```
fuzzyLogicDesigner
```

Fuzzy Logic Designer открывает и отображает схему нечеткой системы вывода с именами каждой входной переменной слева и те из каждой выходной переменной справа, как показано на следующем рисунке. Демонстрационные функции принадлежности, показанные в полях, являются только значками и не изображают фактические формы функций принадлежности.



Ниже схемы имя системы и тип используемого вывода.



В этом примере вы используете Mamdani-вывод-типа по умолчанию. Другой тип вывода, названного Sugeno-выводом-типа, также доступен. Для получения дополнительной информации смотрите [Mamdani и Sugeno Fuzzy Inference Systems](#).

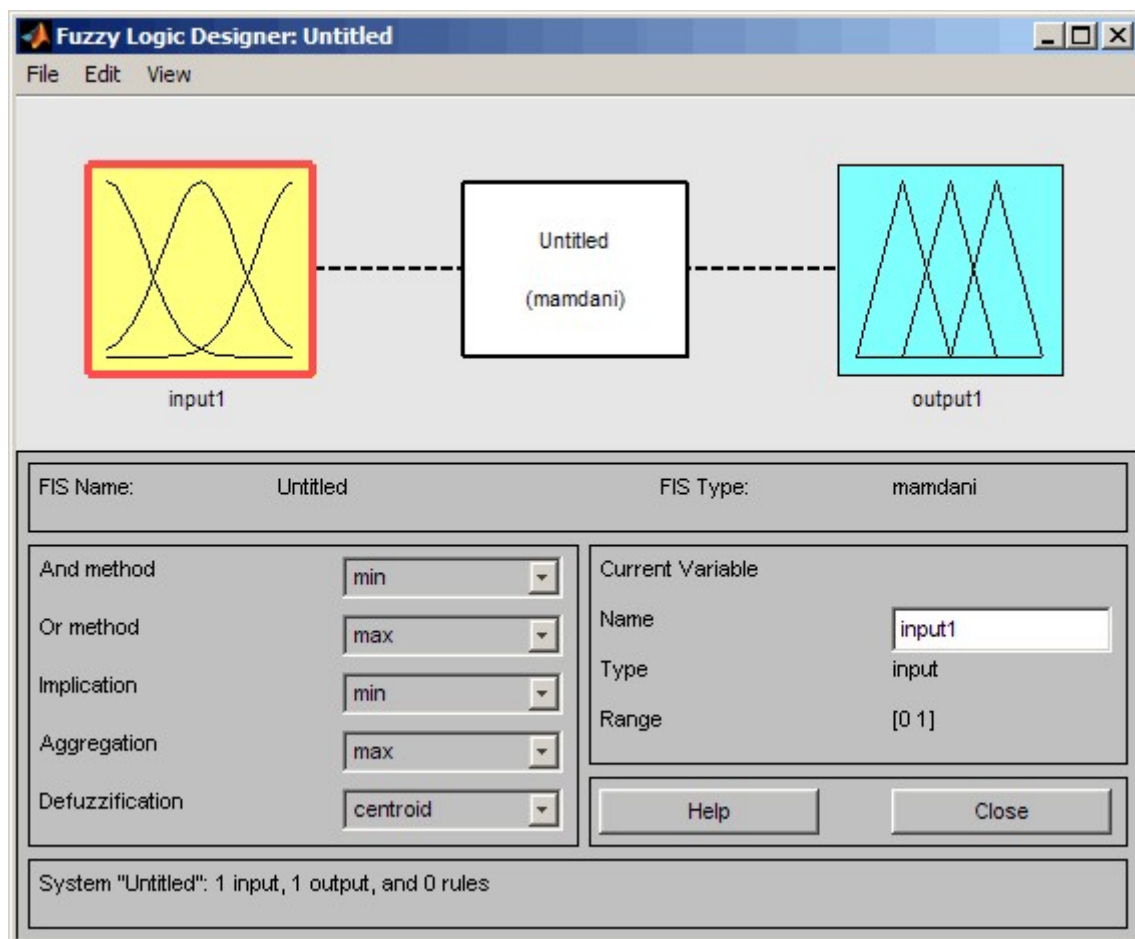
В Fuzzy Logic Designer:

- Выпадающие списки позволяют вам изменить нечеткие функции вывода.
- Область **Current Variable** отображает имя любой переменной ввода или вывода, ее тип и область значений по умолчанию.
- Строка состояния в нижней части отображает информацию о новой операции.

Чтобы создать нечеткую систему вывода, описанную в Базовой задаче расчета чаевых с нуля, введите следующую команду в подсказке MATLAB:

```
fuzzyLogicDesigner
```

Типовой неназванный **Fuzzy Logic Designer** открывается с одним входом **input1**, и один выход **output1**.



Совет

Открыть **Fuzzy Logic Designer** с предварительно созданной нечеткой системой вывода, сохраненной в `tipper.fis`, войти

```
fuzzyLogicDesigner('tipper.fis')
```

Однако, если вы загрузите предварительно созданную систему, вы не создадите правила или создадите функции принадлежности.

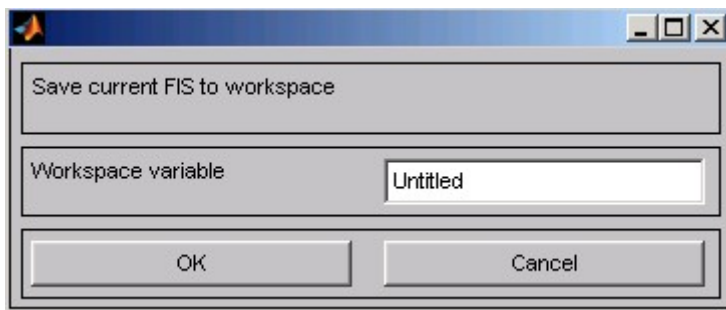
В этом примере вы создаете 2D вход, одну выходную систему. Двумя входными параметрами является **service** и **food**. Тем выход является **tip**.

Добавить вторую входную переменную и изменить имена переменных, чтобы отразить эти обозначения:

1. Выберите **Edit> Add variable> Input**.

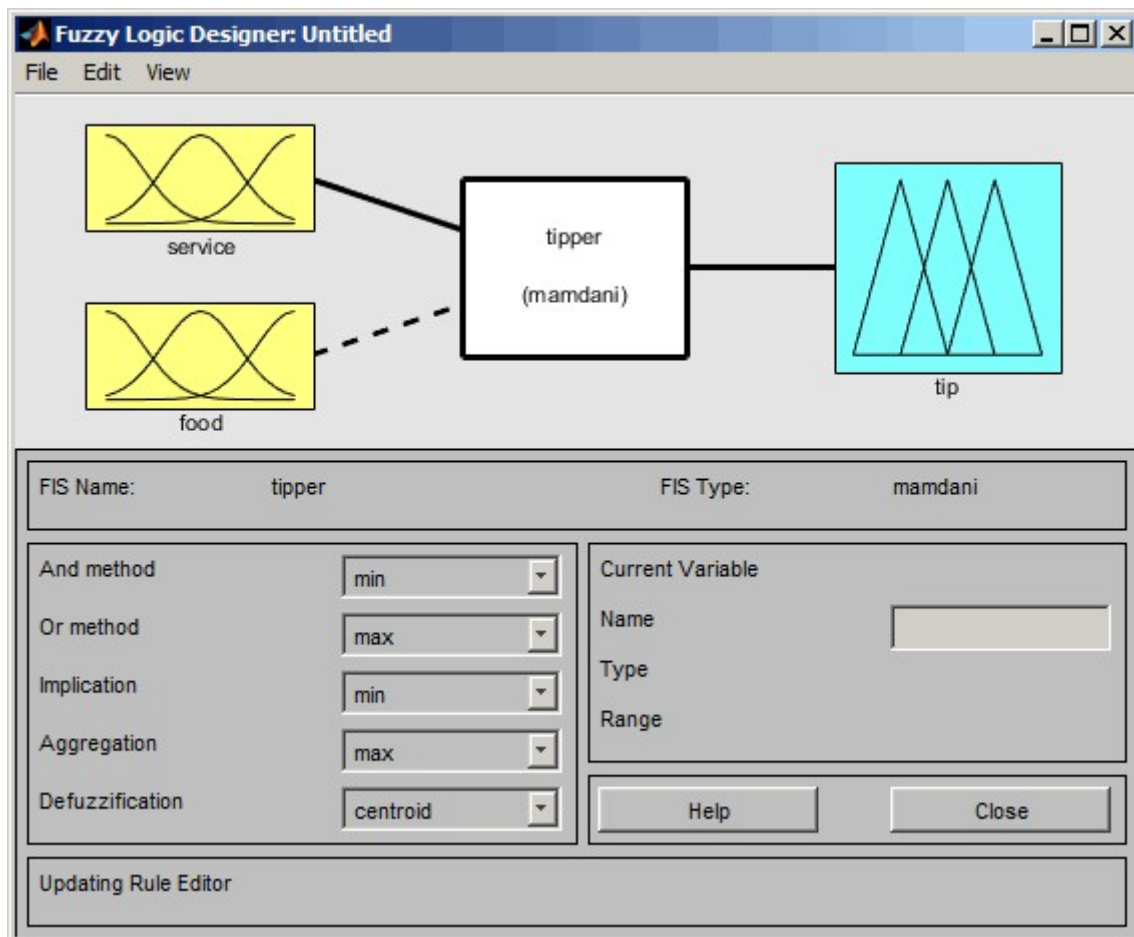
Помеченный **input2** второго желтого поля появляется.

2. Кликните по желтому полю **input1**. Это поле подсвечено с красной схемой.
3. Отредактируйте поле **Name** от input1 к service, и нажмите **Enter**.
4. Кликните по желтому полю **input2**. Это поле подсвечено с красной схемой.
5. Отредактируйте поле **Name** от input2 к food, и нажмите **Enter**.
6. Кликните по синему полю **output1**.
7. Отредактируйте поле **Name** от output1 к tip, и нажмите **Enter**.
8. Выберите **File> Export> To Workspace**.



9. Введите имя **Workspace variable** tipper, и нажмите **OK**.

Схема обновляется, чтобы отразить новые имена переменных ввода и вывода. Существует теперь новая переменная в рабочей области под названием tipper это содержит всю информацию об этой системе. Путем сохранения в рабочую область с новым именем вы также переименовываете целую систему. Ваше окно смотрит что-то как следующая схема.



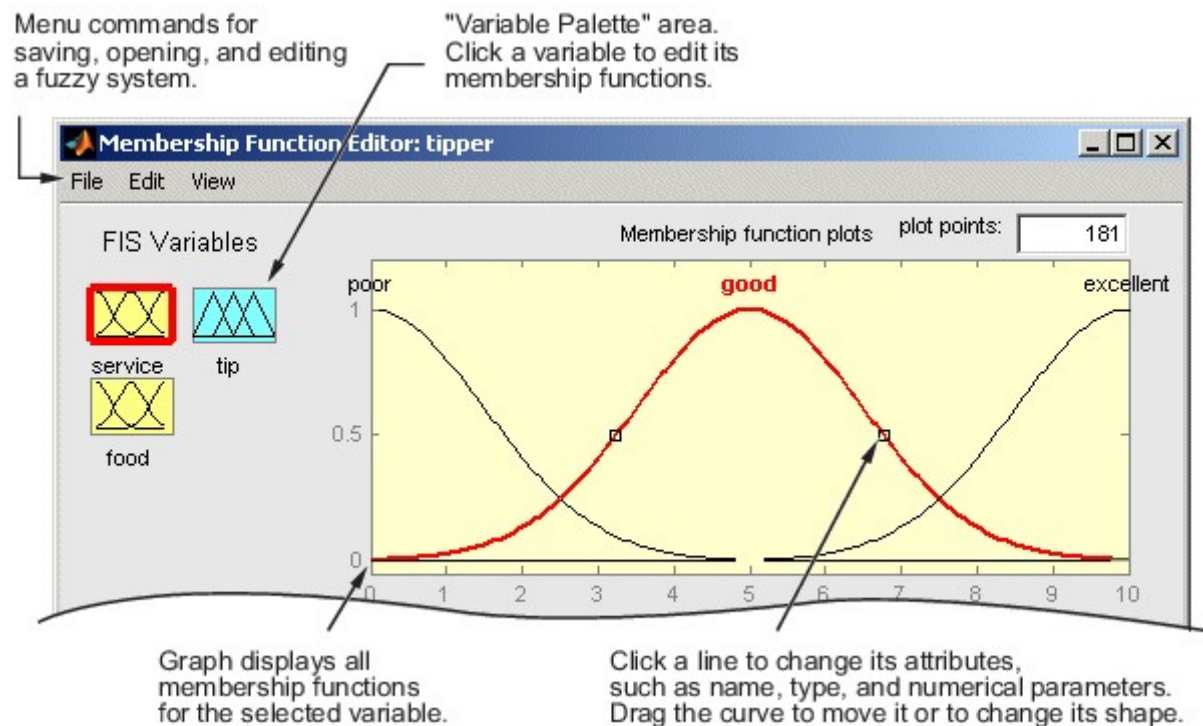
Оставьте опции вывода в нижнем левом углу в их положениях по умолчанию на данный момент. Вы ввели всю информацию, в которой вы нуждаетесь для этого конкретного пользовательского интерфейса. Затем задайте функции принадлежности, сопоставленные с каждой из переменных. Для этого откройте Редактор Функции принадлежности.

Можно открыть Редактор Функции принадлежности одним из трех способов:

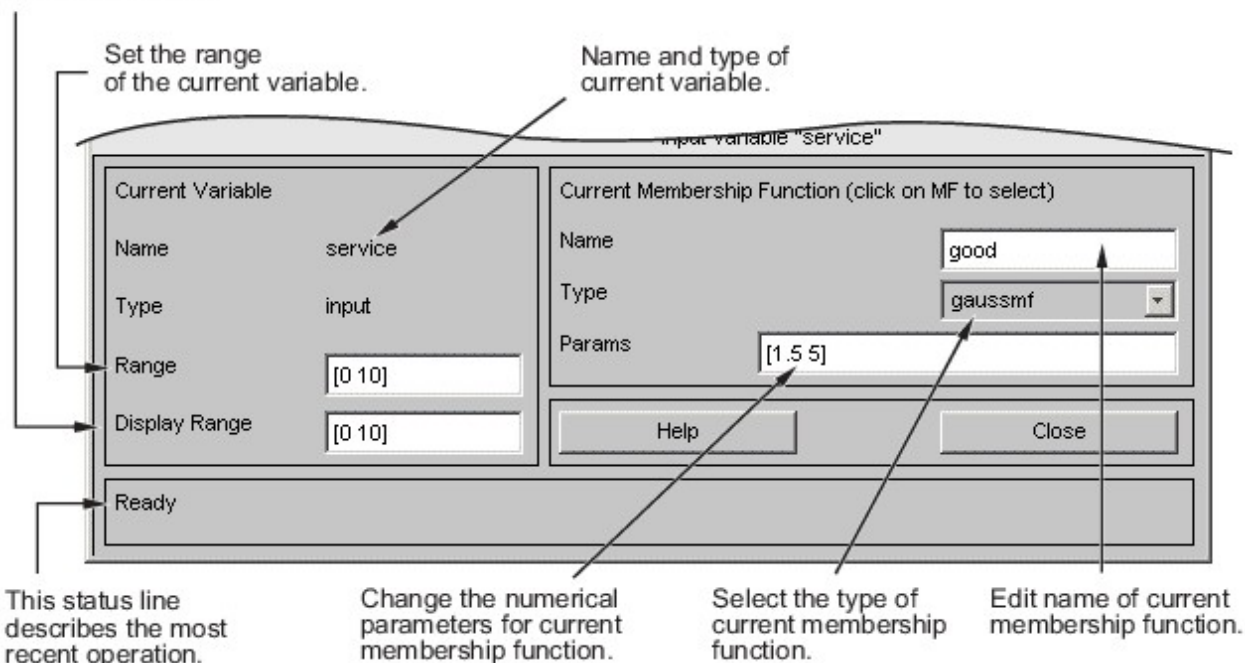
- В рамках приложения **Fuzzy Logic Designer** выберите **Edit> Membership Functions**.
- В рамках приложения **Fuzzy Logic Designer** дважды кликните, синий значок вызвал **совет**.
- В командной строке введите **mfedit**.

Редактор функции принадлежности

Редактор Функции принадлежности является инструментом, который позволяет вам отобразить и отредактировать все функции принадлежности, сопоставленные со всеми переменными ввода и вывода для целой нечеткой системы вывода. Редактор Функции принадлежности совместно использует некоторые функции с **Fuzzy Logic Designer**, как показано на следующем рисунке. На самом деле все пять основных инструментов UI имеют подобные пункты меню, строки состояния и кнопки **Help** и **Close**.



Set the display range of the current plot.



Когда вы открываете Редактор Функции принадлежности, чтобы работать над нечеткой системой вывода, которая уже не существует в рабочей области, нет никаких функций принадлежности, сопоставленных с переменными, которые вы задали с **Fuzzy Logic Designer**.

На верхней левой стороне области графика в Функции принадлежности Редактор является "Переменной Палитрой", которая позволяет вам установить функции принадлежности для данной переменной.

Чтобы настроить функции принадлежности, сопоставленные с входом или выходной переменной для FIS, выберите переменную FIS в этой области путем нажатия на него.

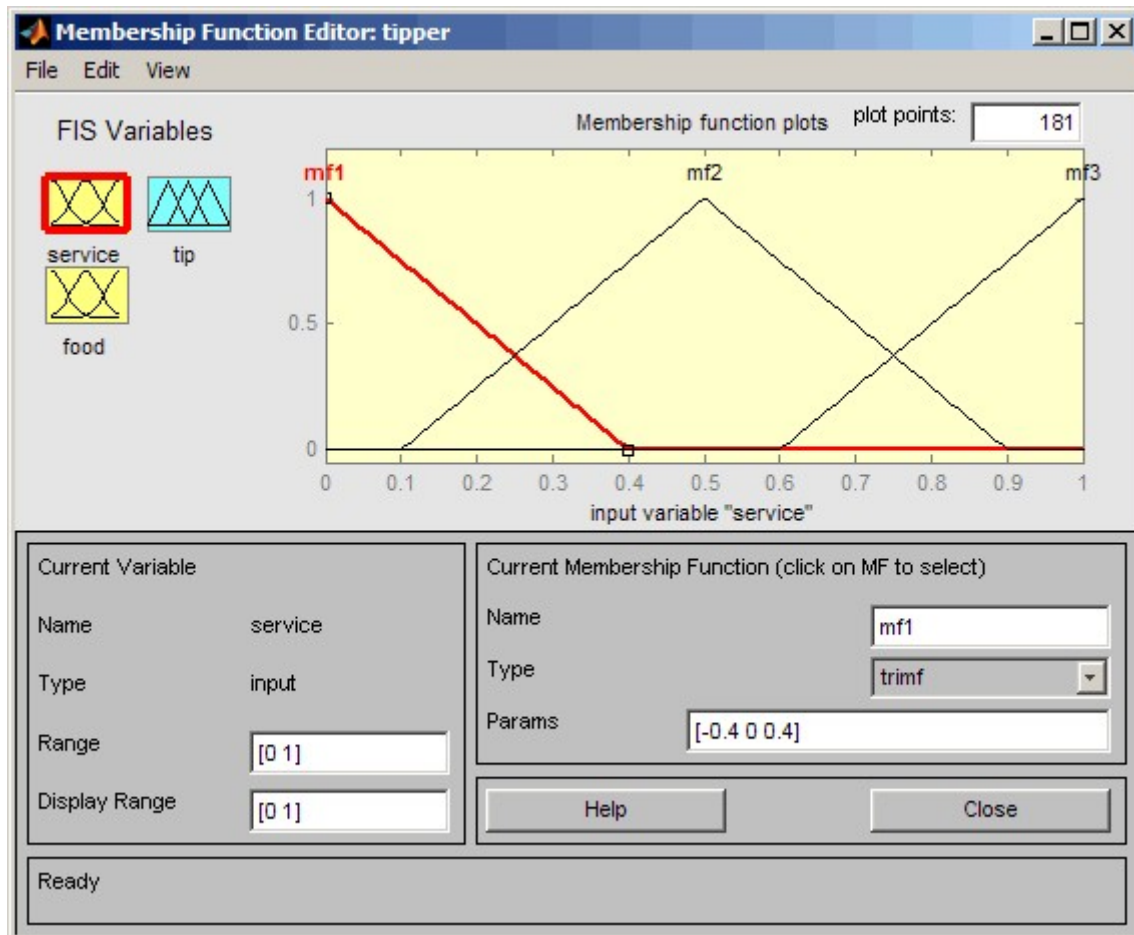
Затем выберите выпадающее меню **Edit** и выберите **Add MFs ..** Новое окно появляется, который позволяет, вы, чтобы выбрать и тип функции принадлежности и количество функций принадлежности сопоставили с выбранной переменной. В нижнем правом углу окна средства управления, которые позволяют вам поменять имя, ввести, и параметры (форма), функции принадлежности, после того, как это выбрано.

Функции принадлежности от текущей переменной отображены в основном графике. Этими функциями принадлежности можно управлять двумя способами. Можно сначала использовать мышь, чтобы выбрать конкретную функцию принадлежности, сопоставленную с данным переменным качеством, (такой как плохую, для переменной, сервиса), и затем перетащить функцию принадлежности поперек. Это действие влияет на математическое описание качества, сопоставленного с той функцией принадлежности для данной переменной. Выбранная функция принадлежности может также быть помечена для расширения, или сокращения путем нажатия на небольшой квадрат перетаскивают точки на функции принадлежности и затем перетаскивание функции с мышью *к внешней стороне*, для расширения, или *к внутренней части*, для сокращения. Это действие изменяет параметры, сопоставленные с той функцией принадлежности.

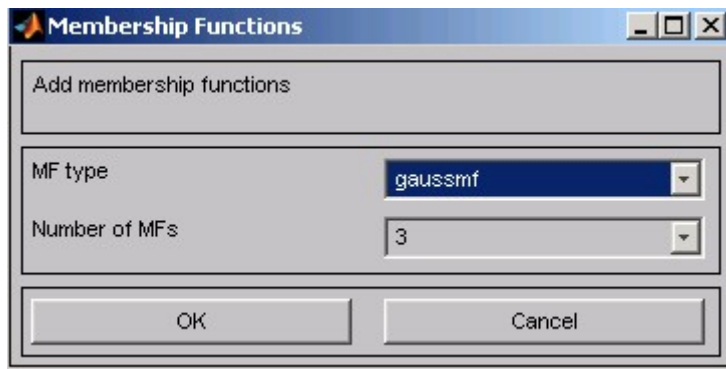
Ниже Переменной Палитры некоторая информация о типе и имени текущей переменной. Существует текстовое поле в этой области, которая позволяет вам изменить пределы области значений текущей переменной (вселенная беседы) и другой, который позволяет вам установить пределы текущего графика (который не оказывает действительного влияния на систему).

Процесс определения функций принадлежности для 2D входного примера добавления подсказки, tipper, следующие:

1. Дважды кликните входную переменную service открыть Редактор Функции принадлежности.



2. В Редакторе Функции принадлежности введите [0 10] в **Области значений** и полях **Display Range**.
3. Создайте функции принадлежности для входной переменной service.
 - a. Выберите **Edit> Remove All MFs**, чтобы удалить функции принадлежности по умолчанию для входной переменной service.
 - b. Выберите **Edit> Add MFs**, чтобы открыть диалоговое окно Membership Functions.
 - c. В диалоговом окне Membership Functions выберите gaussmf как **MF Type**.



- d. Проверьте тот 3 выбран как **Number of MFs**.
 - e. Нажмите **OK**, чтобы добавить три Кривые Гаусса во входную переменную service.
4. Переименуйте функции принадлежности для входной переменной service, и задайте их параметры.
- a. Нажмите на кривую под названием mf1 выбрать его и задать следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите roor.
 - В поле **Params** введите [1.5 0].

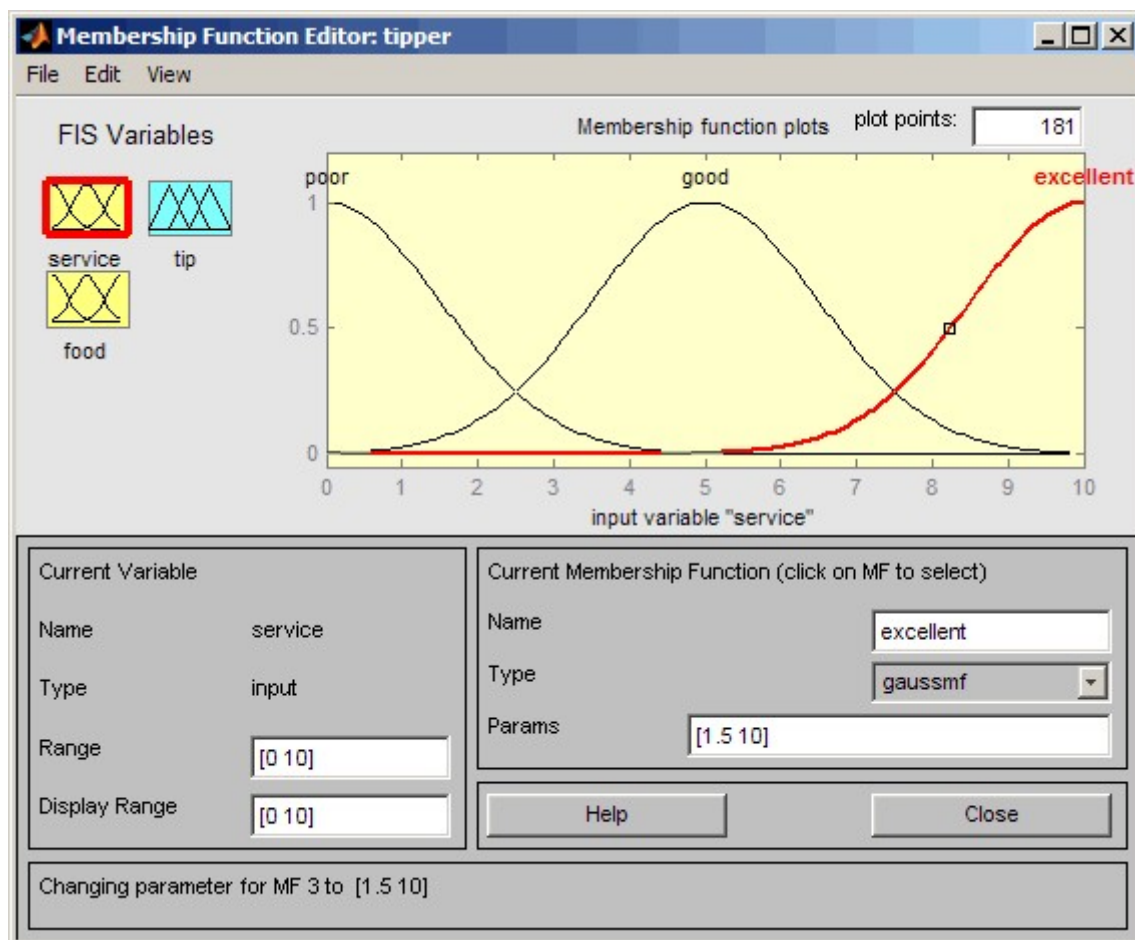
Два входных параметров **Params** представляют стандартное отклонение и сосредотачиваются для Кривой Гаусса.

Совет

Чтобы отрегулировать форму функции принадлежности, введите в желаемых параметрах или используйте мышь, аналогичную описанному ранее.

- b. Нажмите на кривую под названием mf2 выбрать его и задать следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите good.
 - В поле **Params** введите [1.5 5].
- c. Нажмите на кривую под названием mf3, и задайте следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите excellent.
 - В поле **Params** введите [1.5 10].

Окно Membership Function Editor выглядит похожим на следующую фигуру.



5. В области **FIS Variables** кликните по входной переменной food выбрать его.
6. Введите [0 10] в **Области значений** и полях **Display Range**.
7. Создайте функции принадлежности для входной переменной food.
 - a. Выберите **Edit> Remove All MFs**, чтобы удалить Функции принадлежности по умолчанию для входной переменной food.
 - b. Выберите **Edit> Add MFs**, чтобы открыть диалоговое окно Membership Functions.
 - c. В диалоговом окне Membership Functions выберите trapmf как **MF Type**.
 - d. Выберите 2 в **Number of MFs** выпадающий список.
 - e. Нажмите **OK**, чтобы добавить две трапециевидных кривые во входную переменную food.

8. Переименуйте функции принадлежности для входной переменной food, и задайте их параметры:
- В области **FIS Variables** кликните по входной переменной food выбрать его.
 - Нажмите на кривую под названием mf1, и задайте следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите rancid.
 - В поле **Params** введите [0 0 1 3].
 - Нажмите на кривую под названием mf2 выбрать его и ввести delicious в поле **Name**.

Сбросьте связанные параметры при желании.

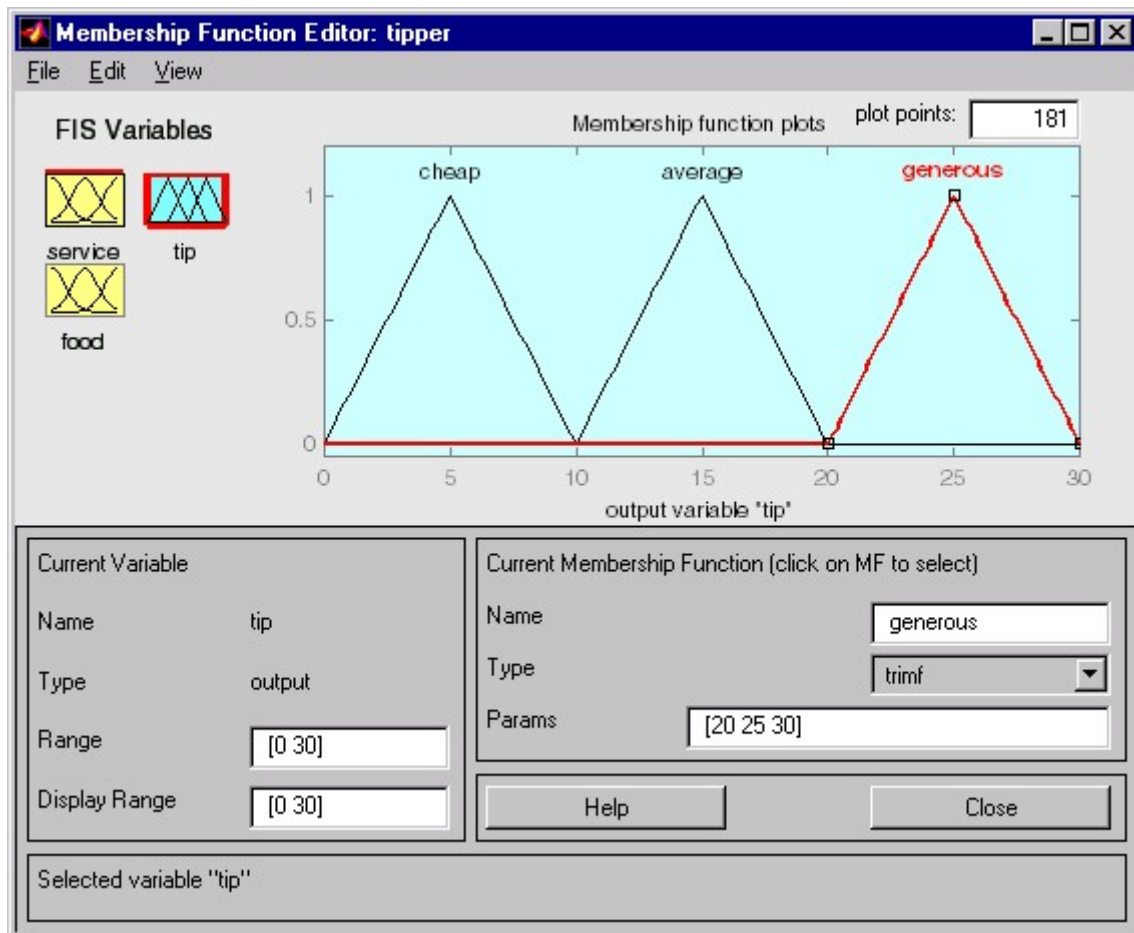
9. Нажмите на выходную переменную tip выбрать его.
10. Введите [0 30] в **Области значений** и полях **Display Range**, чтобы покрыть выходную область значений.

Входные диапазоны от 0 до 10, но выход совет между 5% и 25%.

11. Переименуйте треугольные функции принадлежности по умолчанию для выходной переменной tip, и задайте их параметры.
- Кликните по кривой под названием mf1 выбрать его и задать следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите cheap.
 - В поле **Params** введите [0 5 10].
 - Кликните по кривой под названием mf2 выбрать его и задать следующие поля в области **Current Membership Function (click on MF to select)**:
 - В поле **Name** введите average.
 - В поле **Params** введите [10 15 20].
 - Кликните по кривой под названием mf3 выбрать его и задать следующее:

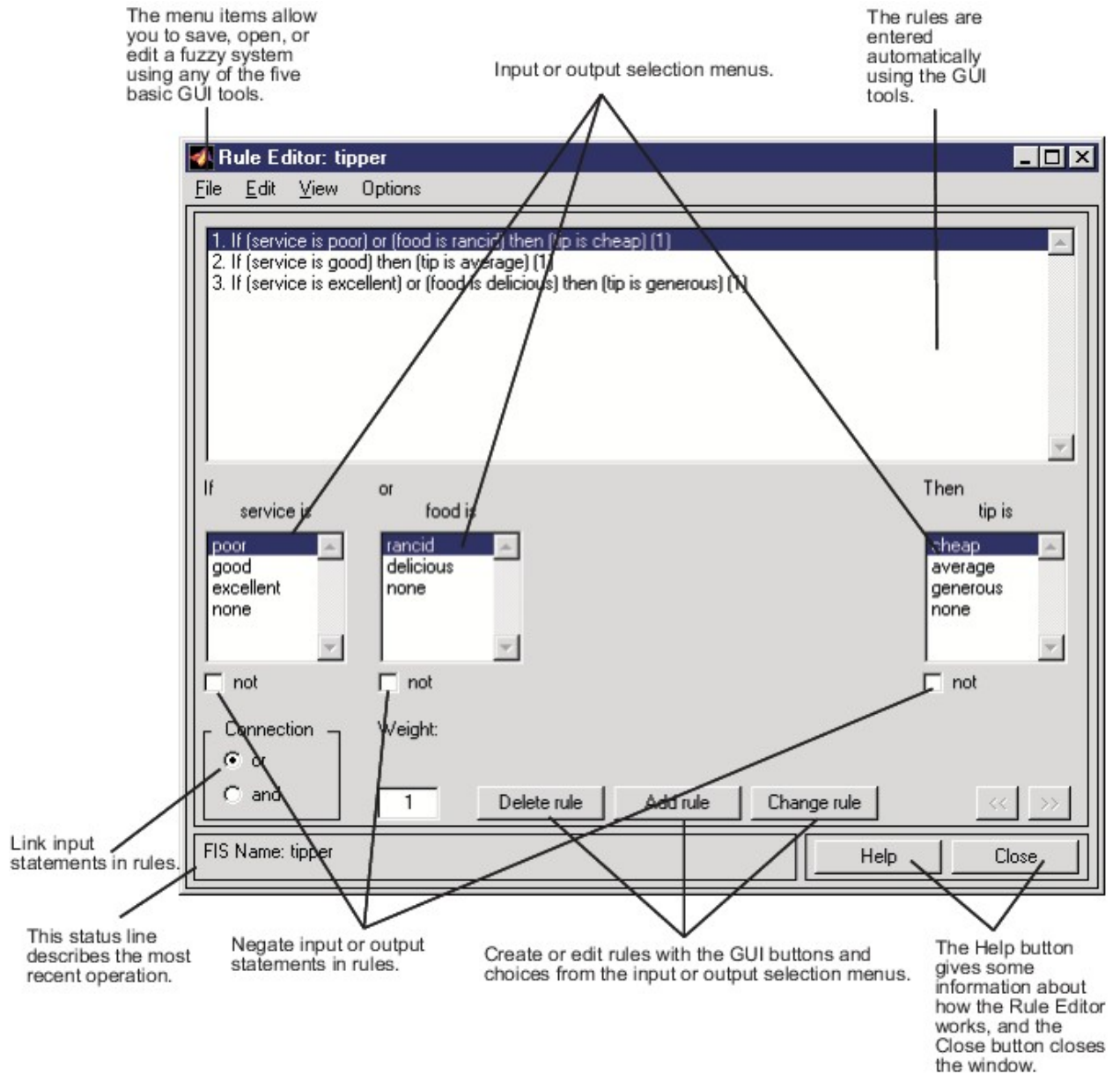
- В поле **Name** введите generous.
- В поле **Params** введите [20 25 30].

Редактор Функции принадлежности выглядит похожим на следующую фигуру.



Теперь, когда переменные назвали, и функции принадлежности имеют соответствующие формы и имена, можно ввести правила. К призыву Редактор Правила войдите в меню **Edit** и выберите **Rules** или введите ruleedit в командной строке.

Редактор правила



Построение правил с помощью графического интерфейса Rule Editor довольно самоочевидно. На основе описаний переменных ввода и вывода, заданных с **Fuzzy Logic Designer**, Редактор Правила позволяет вам создавать операторы правила автоматически. Вы можете:

- Создайте правила путем выбора элемента в каждом поле переменной ввода и вывода, выбора одного элемента **Connection** и нажатия **на Add Rule**. Можно выбрать none как одно из переменных качеств, чтобы исключить ту переменную из данного правила и выбрать not под любым именем переменной, чтобы инвертировать связанное качество.
- Удалите правило путем выбора правила и нажатия **на Delete Rule**.
- Отредактируйте правило путем изменения выбора в переменном поле и нажатия **на Change Rule**.
- Задайте вес к правилу путем ввода в желаемом номере между 0 и 1 в **Weight**. Если вы не задаете вес, он принят, чтобы быть единицей (1).

Подобно тем в **Fuzzy Logic Designer** и Редакторе Функции принадлежности, Редактор Правила имеет панель меню и строку состояния. Пункты меню позволяют вам открывать, закрывать, сохранять и редактировать нечеткую систему с помощью пяти основных инструментов UI. В меню вы можете также:

- Установите формат для отображения путем выбора **Options > Format**.
- Установите язык путем выбора **Options> Language**.

Можно получить доступ к информации о Редакторе Правила путем нажатия на **Help** и закрыть пользовательский интерфейс с помощью **Close**.

Чтобы вставить первое правило в Редактор Правила, выберите следующее:

- poor под переменным **сервисом**
- rancid под переменной **едой**
- **Или** переключатель, в блоке **Connection**
- cheap, под выходной переменной, **советом**.

Затем нажмите **Add rule**.

Получившееся правило

1. Если (сервис плох), или (еда является прогорклой), затем (совет является дешевым)

Числа в круглых скобках представляют веса.

Выполните подобную процедуру, чтобы вставить вторые и третьи правила в Редактор Правила, чтобы добраться

- 1. Если (сервис плох), или (еда является прогорклой), затем (совет является дешевым)*
- 2. Если (сервис хорош), затем (совет является средним)*
- 3. Если (сервис превосходен), или (еда восхитительна), затем (совет щедр)*

Совет

Чтобы изменить правило, сначала нажмите на правило, которое будет изменено. Затем внесите желаемые изменения в то правило, и затем нажмите **правило Change**. Например, чтобы изменить первое правило в

1. Если (сервис, не плохой) или (еда, не прогорклая) затем (совет не является дешевым)

Установите **не** флажок под каждой переменной, и затем нажмите **правило Change**.

Всплывающее меню **Format** из **меню параметров** указывает, что вы смотрите на многословную форму правил. Попробуйте изменить его в symbolic. Вы будете видеть

1. (сервис == плохой) | (еда == прогорклый) => (tip=cheap)

2. (сервис == хороший) => (tip=average)

3. (сервис == превосходный) | (еда == восхитительный) => (tip=generous)

Нет большого различия в отображении действительно, но это - немного больше нейтрального языка, потому что это не зависит от терминов как *если* и *затем*. Если вы изменяете формат в индекслируемый, вы видите чрезвычайно сжатую версию правил.

1 1, 1 (1) : 2

2 0, 2 (1) : 1

3 2, 3 (1) : 2

Это - версия правил, что машина имеет дело с.

- Первый столбец в этой структуре соответствует входным переменным.
- Второй столбец соответствует выходной переменной.
- Третьи отображения столбца вес применились к каждому правилу.
- Четвертый столбец является сокращением, которое указывает, является ли это OR (2) правило или AND (1) правило.

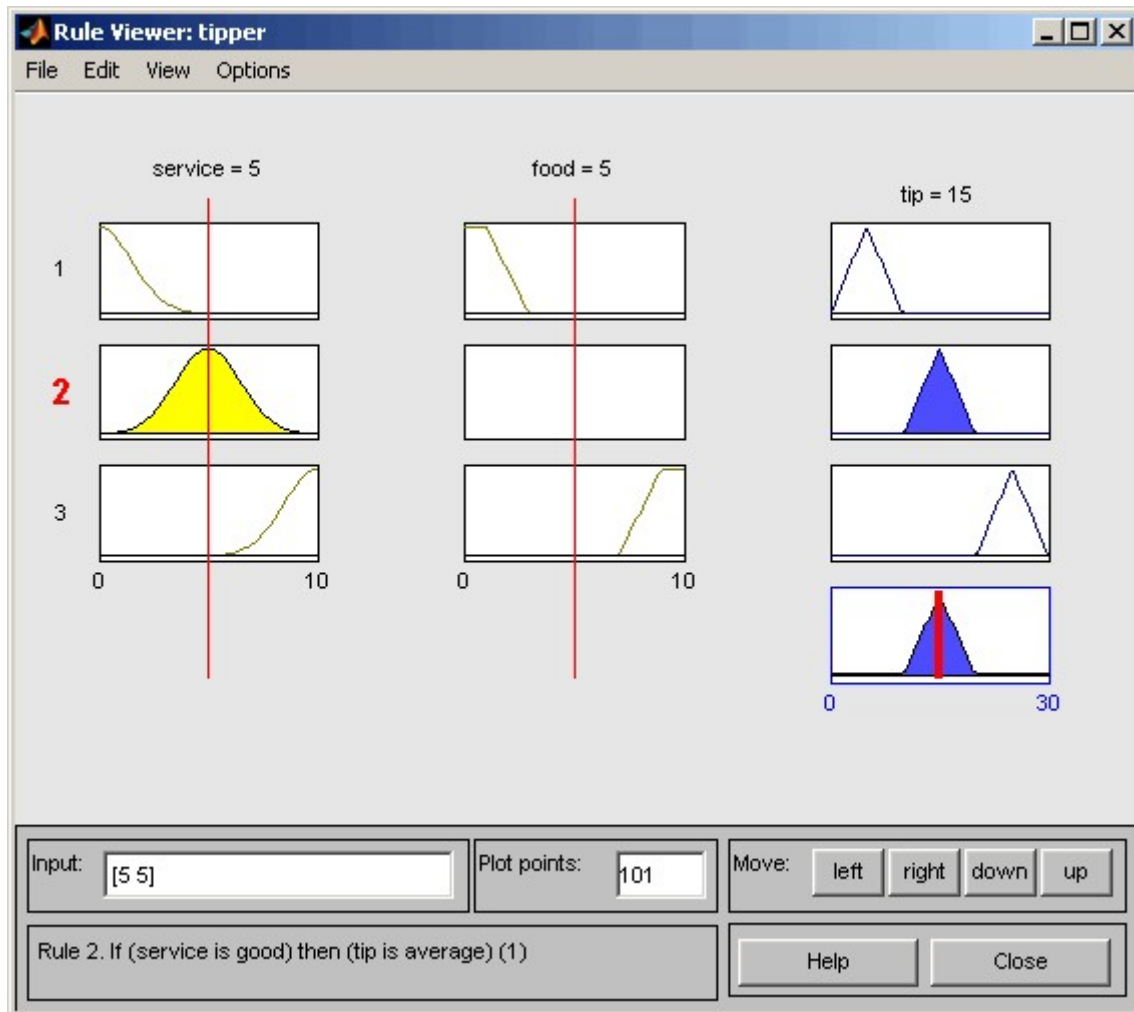
- Числа в первых двух столбцах относятся к индексу функции принадлежности.

Литеральная интерпретация правила 1, "Если введено 1, MF1 (первая функция принадлежности, сопоставленная с входом 1), или, если введено 2 MF1, затем выводит 1, должен быть MF1 (первая функция принадлежности, сопоставленная с выходом 1) с весом 1".

Символьный формат не рассматривает члены, *если, то*, и так далее. Индексируемый формат даже не беспокоится именами ваших переменных. Очевидно, функциональность вашей системы не зависит от того, как хорошо вы назвали свои переменные и функции принадлежности. Смысл именования переменных описательно, как всегда, делая систему легче для вас интерпретировать. Таким образом, если вы не имеете некоторое особое назначение в виду, это, вероятно, быть легче для вас продолжить **многословный** формат.

На данном этапе нечеткая система вывода была полностью задана, в этом переменные, функции принадлежности, и правила, необходимые, чтобы вычислить советы, существуют. Теперь посмотрите на нечеткую схему вывода, представленную в конце предыдущего раздела, и проверьте, что все ведет себя способ, которым вы думаете, что это должно. Можно использовать Средство просмотра Правил, следующие из инструментов UI, на которые мы посмотрим. В меню **View** выберите **Rules**.

Средство просмотра правила



Средство просмотра Правила отображает дорожную карту целого нечеткого процесса вывода. Это основано на нечеткой схеме вывода, описанной в предыдущем разделе. Вы видите окно одной фигуры с 10 графиками, вложенными в нем. Три графика через верхнюю часть фигуры представляют antecedent и следствие первого правила. Каждое правило является строкой графиков, и каждый столбец является переменной. Числа правила отображены слева от каждой строки. Можно нажать на номер правила, чтобы просмотреть правило в строке состояния.

- Первые два столбца графиков (шесть желтых графиков) показывают функции принадлежности, на которые ссылается antecedent или если-часть каждого правила.
- Третий столбец графиков (три синих графика) показывает функции принадлежности, на которые ссылается следствие или тогдашняя часть каждого правила.

Заметьте, что под **едой**, существует график, который является пробелом. Это соответствует характеристике *none* для переменной **еды** во втором правиле.

- Четвертый график в третьем столбце графиков представляет взвешенное решение агрегата для данной системы вывода.

Это решение будет зависеть от входных значений для системы.

defuzzified выход отображен как полужирная вертикальная линия на этом графике.

Переменные и их текущие значения отображены сверху столбцов. В нижнем левом углу существует текстовое поле **Input**, в которое можно ввести определенные входные значения. Для 2D входной системы вы введете входной вектор, [9 8], например, и затем нажмете **Enter**. Можно также настроить эти входные значения путем нажатия на любой из трех графиков для каждого входа. Это переместит красную линию индекса горизонтально к точке, где вы щелкнули. В качестве альтернативы можно также перетащить эту линию для того, чтобы изменить входные значения. Когда вы выпускаете линию, (или после ручного определения входа), новое вычисление выполняется, и вы видите, что целый нечеткий процесс вывода происходит:

- То, где сервис представления линии индекса пересекается, сервис "линии функции принадлежности плох" в верхнем левом графике, определяет степень, до которого правила каждый активируется.
- Желтая закрашенная фигура цвета под фактической кривой функции принадлежности используется, чтобы сделать нечеткое значение членства визуально очевидным.

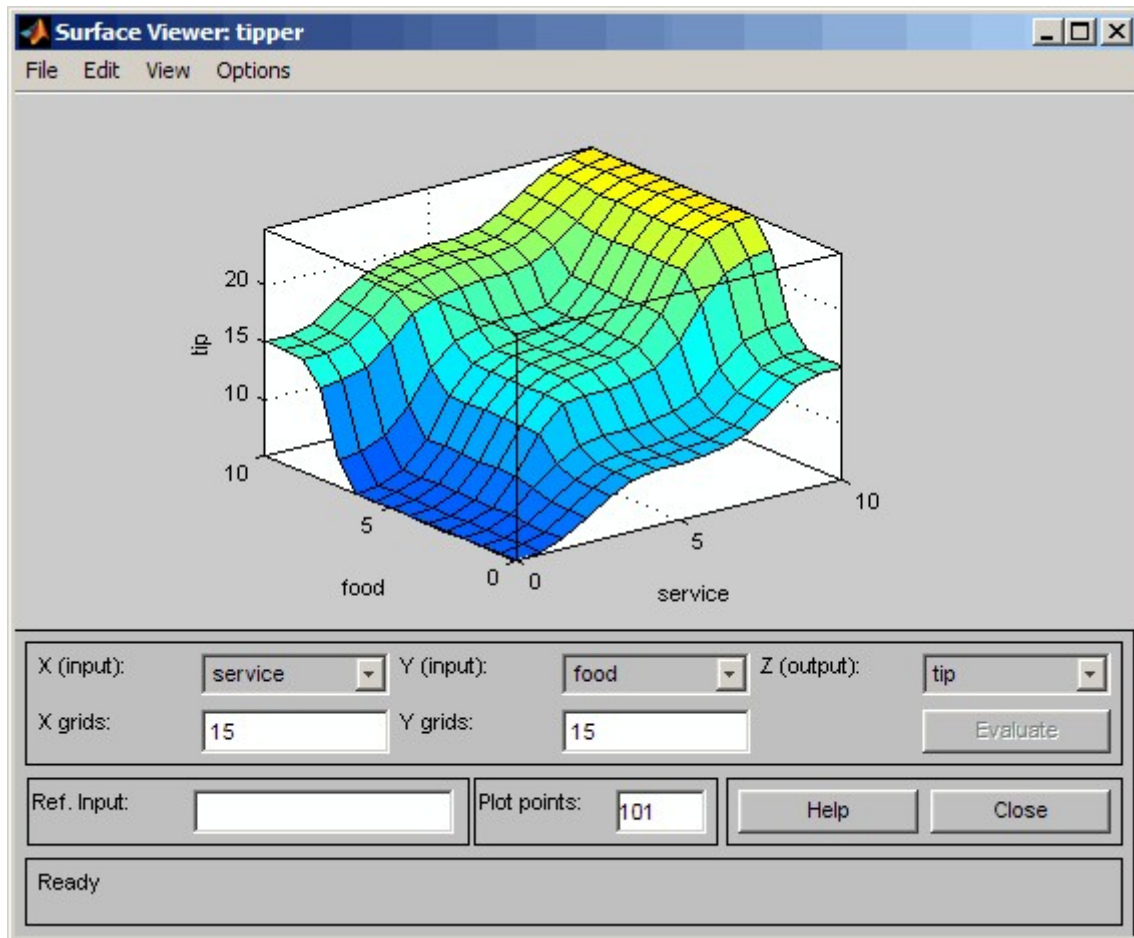
Каждая из характеристик каждой из переменных задана относительно входной линии индекса этим способом. Если вы следуете правилу 1 через верхнюю часть схемы, вы видите, что последовательный "совет является дешевым", было усеченным до точно той же степени как (составной) антецедент — это - процесс значения в действии. Агрегация происходит вниз третий столбец, и результирующий совокупный график показывают в одном графике, появляющемся в правом нижнем углу поля графика. defuzzified выходное значение показывает толстая линия, проходящая через совокупное нечеткое множество.

Можно переключить графики с помощью **left**, **right**, **down** и **up**. Пункты меню позволяют вам сохранять, открывать, или редактировать нечеткую систему с помощью любого из пяти основных инструментов UI.

Средство просмотра Правила позволяет вам интерпретировать целый нечеткий процесс вывода целиком. Средство просмотра Правила также показывает, как форма определенных функций принадлежности влияет на полный результат. Поскольку это строит каждую часть каждого правила, это может стать громоздким для особенно больших систем, но, для относительно небольшого количества вводов и выводов, это выполняет хорошо (в зависимости от того, сколько экранного места вы отводите для него) максимум с 30 правилами и целых 6 или 7 переменными.

Средство просмотра Правила показывает одно вычисление за один раз и очень подробно. В этом смысле это представляет своего рода микро представление нечеткой системы вывода. Если вы хотите видеть целую выходную поверхность своей системы — целый промежуток выходного набора на основе целого промежутка входного набора — необходимо открыть Поверхностное Средство просмотра. Это средство просмотра является последним из пяти основных инструментов Fuzzy Logic Toolbox UI. Чтобы открыть Поверхностное Средство просмотра, выберите **Surface** из меню **View**.

Поверхностное средство просмотра



После открытия Поверхностного Средства просмотра вы видите 3D кривую, которая представляет отображение от качества продуктов и качества обслуживания, чтобы снабдить подсказкой сумму. Поскольку эта кривая представляет 2D вход случай с одним выходом, вы видите, что целое сопоставляет в одном графике. Когда мы перемещаемся вне трех измерений в целом, мы начинаем сталкиваться с проблемой, отображающей результаты.

Соответственно, Поверхностное Средство просмотра оборудовано выпадающими меню **X (input)**, **Y (input)** и **Z (output)**, которые позволяют вам выбрать любые два входных параметров и любой выход для графического вывода. Ниже этих меню два поля ввода **X grids** и **Y grids**, которые позволяют вам задать, сколько осей X и линий сетки оси Y вы хотите включать. Эта возможность позволяет вам сохранять время вычисления разумным для комплексных проблем.

По умолчанию объемная поверхностная диаграмма обновляется автоматически, когда вы изменяете выборы переменной ввода или вывода или количество узлов решетки. Чтобы отключить автоматические обновления графика, в меню **Options**, очищают опцию **Always evaluate**. Когда эта опция будет отключена, чтобы обновить график, нажмите **Evaluate**.

Если вы хотите создать более сглаженный график, используйте поле **Plot points**, чтобы задать число точек, на котором функции принадлежности оценены в области значений ввода или вывода. Это поле значения по умолчанию к минимальному количеству графиков графика, 101. Если вы задаете меньше точек графика, значение поля автоматически сбрасывает к 101. Когда вы задаете количество точек графика, объемная поверхностная диаграмма автоматически обновляется.

Путем нажатия на оси графика и перетаскивания мыши, можно управлять поверхностью так, чтобы можно было просмотреть его от различных углов.

Касательно Поля ввода используется в ситуациях, когда существует больше входных параметров, требуемых системой, чем поверхность сопоставляет. Можно отредактировать это поле, чтобы явным образом установить входные параметры, не заданные в объемной поверхностной диаграмме.

Предположим, что вы имеете систему с одним выходом с четырьмя входами и хотели бы видеть, что выход появляется. Поверхностное Средство просмотра может сгенерировать 3D выходную поверхность, где любые два из входных параметров варьируются, но два из входных параметров должны считаться постоянными, потому что компьютерные мониторы не могут отобразить пятимерную форму. В таком случае вход является четырехмерным вектором с NaNs содержание места различных входных параметров, в то время как численные значения указывает на те значения, которые остаются фиксированными.

Пункты меню позволяют вам открывать, закрывать, сохранять и редактировать нечеткую систему с помощью пяти основных инструментов UI. Можно получить доступ к информации о Поверхностном Средстве просмотра путем нажатия **на Help** и закрыть пользовательский интерфейс с помощью **Close**.

Импортирование и экспортирование нечетких систем вывода

Когда вы сохраняете нечеткую систему в файл, вы сохраняете текст ASCII, представление файла FIS той системы с файлом снабжает суффиксом .fis. Вручную не редактируйте содержимое .fis файл. Выполнение так может привести к неожиданным результатам при загрузке файла. Когда вы сохраняете свою нечеткую систему в рабочее пространство MATLAB, вы создаете переменную, которая действует как объект MATLAB для нечеткой системы.

Примечание

Если вы не сохраняете свой FIS в файл, но только сохраняете его в рабочее пространство MATLAB, вы не можете восстановить его для использования в новом сеансе работы с MATLAB.

Создайте нечеткие системы в командной строке

Можно создать нечеткую систему вывода (FIS) в командной строке MATLAB®. Этот метод является альтернативой интерактивной разработке вашего FIS использование [Fuzzy Logic Designer](#).

Этот пример показывает вам, как создать Mamdani нечеткая система вывода. В то время как вы создаете Mamdani FIS, используемые методы применяются к созданию систем Sugeno также.

Задача расчета чаевых в командной строке

Чтобы продемонстрировать функциональность командной строки для создания и просмотра нечетких систем вывода, этот пример использует самосвал FIS.

```
fis = readfis('tipper.fis');
```

Эта команда возвращает mamfis объект, который содержит свойства нечеткой системы. Для системы Sugeno эта команда возвращает sugfis объект.

Можно получить доступ к свойствам FIS с помощью записи через точку. Например, просмотрите входные параметры нечеткой системы.

```
fis.Inputs
```

```
ans =
```

```
1x2 fisvar array with properties:
```

```
Name
```

```
Range
```

```
MembershipFunctions
```

```
Details:
```

	Name	Range	MembershipFunctions
	_____	_____	_____

1	"service"	0 10	{1x3 fismf}
---	-----------	------	-------------

2	"food"	0 10	{1x2 fismf}
---	--------	------	-------------

Чтобы установить свойства вашей нечеткой системы, используйте запись через точку. Например, определите имя FIS.

```
fis.Name = "gratuity";
```

Объект FIS

Вы представляете нечеткие системы вывода с помощью [mamfis](#) и [sugfis](#) объекты. Эти объекты содержат всю нечеткую информацию о системе вывода, включая имена переменных, определения функции принадлежности и нечеткие методы вывода. Каждый FIS является самостоятельно иерархией объектов. Следующие объекты используются в нечеткой системе:

- [fisvar](#) объекты представляют обе переменные ввода и вывода.
- [fismf](#) объекты представляют функции принадлежности в каждой переменной ввода и вывода.
- [fisrule](#) объекты представляют нечеткие правила, что карта вводит к выходным параметрам.

Просмотрите всю информацию для FIS путем прямого листинга его свойств.

`fis`

`fis =`

`mamfis with properties:`

`Name: "gratuity"`

`AndMethod: "min"`

`OrMethod: "max"`

`ImplicationMethod: "min"`

`AggregationMethod: "max"`

`DefuzzificationMethod: "centroid"`

`Inputs: [1x2 fisvar]`

`Outputs: [1x1 fisvar]`

`Rules: [1x3 fisrule]`

`DisableStructuralChecks: 0`

See 'getTunableSettings' method for parameter optimization.

Можно посмотреть свойства объектов в объекте FIS с помощью записи через точку. Например, посмотрите fisvar объект для первой входной переменной.

```
fis.Inputs(1)
```

```
ans =
```

fisvar with properties:

Name: "service"

Range: [0 10]

MembershipFunctions: [1x3 fismf]

Кроме того, посмотрите функции принадлежности для этой переменной.

```
fis.Inputs(1).MembershipFunctions
```

```
ans =
```

1x3 fismf array with properties:

Type

Parameters

Name

Details:

Name	Type	Parameters
------	------	------------

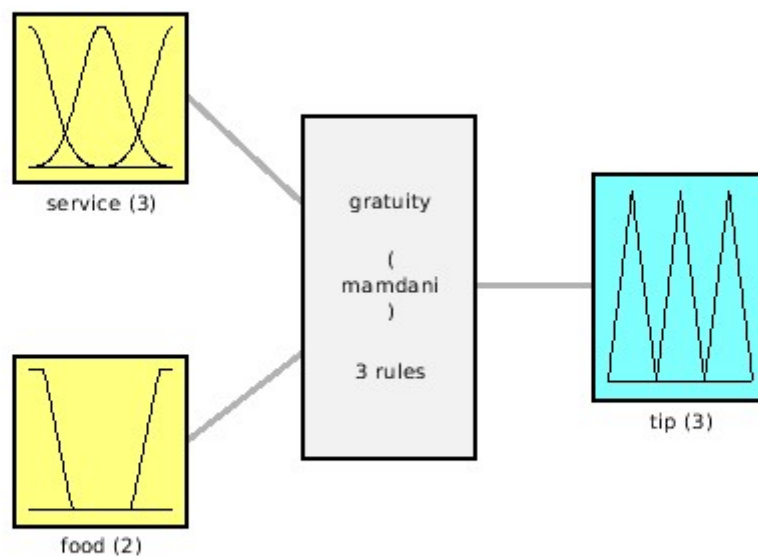
_____	_____	_____
-------	-------	-------

1	"poor"	"gaussmf"	1.5	0
2	"good"	"gaussmf"	1.5	5
3	"excellent"	"gaussmf"	1.5	10

Системные функции отображения

Чтобы получить высокоуровневое представление вашей нечеткой системы из командной строки, используйте `plotfis`, `plotmf`, и `gensurf` функции. `plotfis` отображает целую систему как блок-схему, как показано в **Fuzzy Logic Designer**.

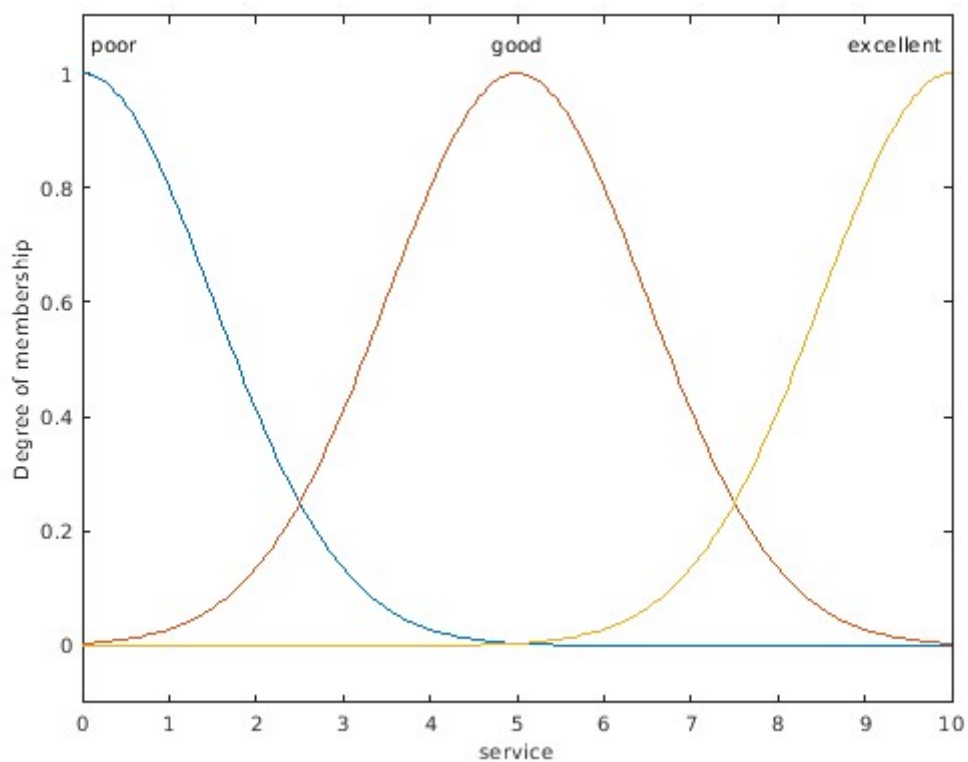
`plotfis(fis)`



System gratuity: 2 inputs, 1 outputs, 3 rules

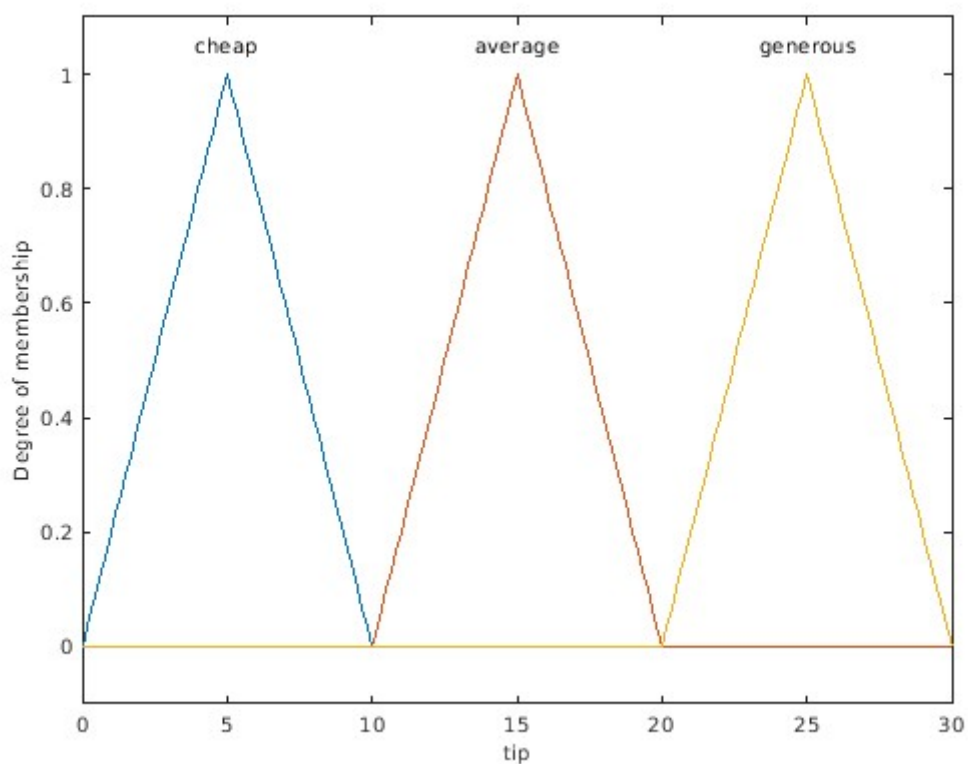
`plotmf` графики функций все функции принадлежности сопоставлены с данной переменной. Например, просмотрите функции принадлежности для первой входной переменной.

`plotmf(fis,'input',1)`



Точно так же, чтобы просмотреть функции принадлежности для первого выхода, введите:

```
plotmf(fis,'output',1)
```



plotmf не поддерживает просмотр выходных функций принадлежности для систем Sugeno.

Чтобы просмотреть правила нечеткой системы, введите:

```
fis.Rules
```

```
ans =
```

```
1x3 fisrule array with properties:
```

Description

Antecedent

Consequent

Weight

Connection

Details:

Description

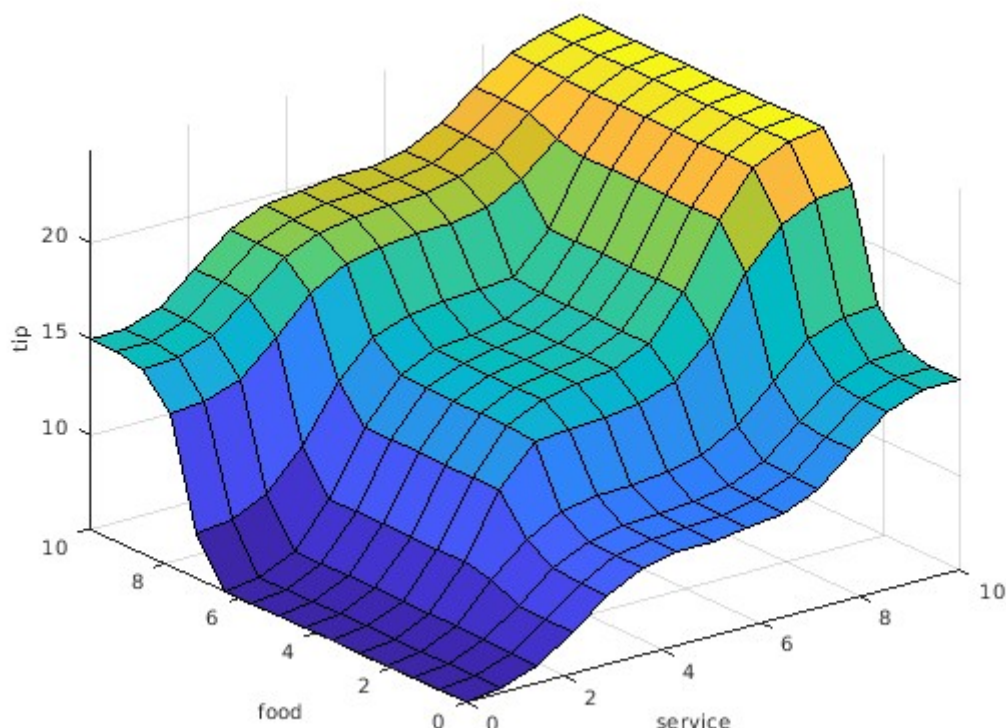
1 "service==poor | food==rancid => tip=cheap (1)"

2 "service==good => tip=average (1)"

3 "service==excellent | food==delicious => tip=generous (1)"

gensurf графики функций выход FIS для любого или двух входных переменных.

```
gensurf(fis)
```



Создайте нечеткую систему вывода

Как альтернатива использованию приложения **Fuzzy Logic Designer**, можно создать FIS полностью из командной строки.

Во-первых, создайте Mamdani FIS, задав его имя.

```
fis = mamfis('Name','tipper');
```

Добавьте первую входную переменную для качества обслуживания с помощью [addInput](#).

```
fis = addInput(fis,[0 10],Name,'service');
```

Добавьте функции принадлежности для каждого из уровней качества обслуживания с помощью [addMF](#). В этом случае используйте Гауссовы функции принадлежности. Для получения дополнительной информации о Гауссовых свойствах функции принадлежности смотрите [gaussmf](#).

```
fis = addMF(fis,"service","gaussmf",[1.5 0],Name,'poor');
```

```
fis = addMF(fis,"service","gaussmf",[1.5 5],Name,'good');
```

```
fis = addMF(fis,"service","gaussmf",[1.5 10],Name,'excellent');
```

Добавьте вторую входную переменную для качества продуктов и добавьте две трапециевидных функции принадлежности. Для получения информации о трапециевидных функциях принадлежности смотрите [trapmf](#).

```
fis = addInput(fis,[0 10],Name,"food");
```

```
fis = addMF(fis,"food","trapmf",[-2 0 1 3],Name,"rancid");
```

```
fis = addMF(fis,"food","trapmf",[7 9 10 12],Name,"delicious");
```

Добавьте выходную переменную для совета и добавьте три треугольных функции принадлежности. Для получения дополнительной информации о треугольной функции принадлежности смотрите [trimf](#).

```
fis = addOutput(fis,[0 30],Name,"tip");
```

```
fis = addMF(fis,"tip","trimf",[0 5 10],Name,"cheap");
```

```
fis = addMF(fis,"tip","trimf",[10 15 20],Name,"average");
```

```
fis = addMF(fis,"tip","trimf",[20 25 30],Name,"generous");
```

Задайте следующие три правила для FIS как числовой массив:

1. Если (сервис плох), или (еда является прогорклой), то (совет является дешевым).
2. Если сервис хорош, то чаевые средние.
3. Если сервис превосходен, или еда восхитительна, то чаевые щедрые.

Каждая строка массива содержит одно правило в следующем формате.

- Столбец 1 - индекс функции принадлежности для первого входа
- Столбец 2 - индекс функции принадлежности для второго входа
- Столбец 3 - индекс функции принадлежности для выхода
- Столбец 4 - вес Правила (от 0 к 1)
- Столбец 5 - Нечеткий оператор (1 для AND, 2 для OR)

Для индексов функции принадлежности укажите на НЕ условие с помощью отрицательной величины. Для получения дополнительной информации о нечеткой спецификации правила смотрите [addRule](#).

```
ruleList = [1 1 1 1 2;  
            2 0 2 1 1;  
            3 2 3 1 2];
```

Добавьте правила в FIS.

```
fis = addRule(fis,ruleList);
```

В качестве альтернативы можно создать нечеткую систему вывода с помощью комбинации записи через точку и `fisvar`, `fismf`, и `fisrule` объекты. Этот метод не является хорошей практикой для большинства приложений. Однако можно использовать этот подход, когда приложение требует большей гибкости в построении и изменении FIS.

Создайте нечеткую систему вывода.

```
fis = mamfis('Name','tipper');
```

Добавьте и сконфигурируйте первую входную переменную. В этом случае создайте `fisvar` по умолчанию, возразите и задайте его свойства с помощью записи через точку.

```
fis.Inputs(1) = fisvar;
```

```
fis.Inputs(1).Name = "service";
```

```
fis.Inputs(1).Range = [0 10];
```

Задайте функции принадлежности для первой входной переменной. Для каждого MF создайте `fismf` объект и набор свойства с помощью записи через точку.

```
fis.Inputs(1).MembershipFunctions(1) = fismf;
```

```
fis.Inputs(1).MembershipFunctions(1).Name = "poor";
```

```
fis.Inputs(1).MembershipFunctions(1).Type = "gaussmf";
```

```
fis.Inputs(1).MembershipFunctions(1).Parameters = [1.5 0];
```

```
fis.Inputs(1).MembershipFunctions(2) = fismf;
```

```
fis.Inputs(1).MembershipFunctions(2).Name = "good";
```

```
fis.Inputs(1).MembershipFunctions(2).Type = "gaussmf";
```

```
fis.Inputs(1).MembershipFunctions(2).Parameters = [1.5 5];
```

```
fis.Inputs(1).MembershipFunctions(3) = fismf;
```

```
fis.Inputs(1).MembershipFunctions(3).Name = "excellent";
```

```
fis.Inputs(1).MembershipFunctions(3).Type = "gaussmf";
```

```
fis.Inputs(1).MembershipFunctions(3).Parameters = [1.5 10];
```

Добавьте и сконфигурируйте вторую входную переменную. Для этой переменной задайте имя и область значений, когда вы создадите `fisvar` объект.

```
fis.Inputs(2) = fisvar([0 10], 'Name', 'food');
```

Задайте функции принадлежности для второго входа. Для каждого MF задайте имя, введите, и параметры, когда вы создадите `fismf` объект.

```
fis.Inputs(2).MembershipFunctions(1) = fismf("trapmf", [-2 0 1 3], ...  
      'Name', 'rancid');
```

```
fis.Inputs(2).MembershipFunctions(2) = fismf("trapmf", [7 9 10 12], ...  
      'Name', 'delicious');
```

Точно так же добавьте и сконфигурируйте выходную переменную и ее функции принадлежности.

```
fis.Outputs(1) = fisvar([0 30], 'Name', 'tip');
```

В этом случае задайте выходные функции принадлежности с помощью вектора из `fismf` объекты.

```
mf1 = fismf("trimf", [0 5 10], 'Name', 'cheap');
```

```
mf2 = fismf("trimf", [10 15 20], 'Name', 'average');
```

```
mf3 = fismf("trimf", [20 25 30], 'Name', 'generous');
```

```
fis.Outputs(1).MembershipFunctions = [mf1 mf2 mf3];
```

Создайте правила для нечеткой системы. Поскольку каждое правило создает `fisrule` объект. Затем задайте правила с помощью вектора из этих объектов. При создании `fisrule` объект с помощью числовых значений, необходимо задать количество входных переменных.

```
rule1 = fisrule([1 1 1 1 2],2);
rule2 = fisrule([2 0 2 1 1],2);
rule3 = fisrule([3 2 3 1 2],2);
rules = [rule1 rule2 rule3];
```

Прежде, чем добавить ваши правила в вашу нечеткую систему, необходимо обновить их использующий данные в объекте FIS. Обновите правила с помощью update функция, и добавляет их нечеткая система.

```
rules = update(rules,fis);
fis.Rules = rules;
```

При построении нечеткой системы можно также задать пользовательские функции принадлежности и заключить функции. Для получения дополнительной информации смотрите [Сборку Нечеткие Системы Используя Пользовательские Функции](#).

Оцените нечеткую систему вывода

Чтобы оценить выход нечеткой системы для данной входной комбинации, используйте [evalfis](#) команда. Например, оцените fis использование значений входной переменной 1 и 2.

```
evalfis(fis,[1 2])
ans = 5.5586
```

Можно также оценить несколько входных комбинаций с помощью массива, где каждая строка представляет входную комбинацию того.

```
inputs = [3 5;
          2 7;
          3 1];
evalfis(fis,inputs)
ans = 3×1
```

7.7885

8.9547